

**DETECTION AND CLASSIFICATION OF FAKE NEWS
USING DEEPLARNING TECHNIQUE E- BERT (EXTENDED
BIDIRECTIONAL ENCODER REPRESENTATIONS FROM
TRANSFORMERS)**

*Project Report submitted
in partial fulfilment for the award of the Degree of*

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

by

K. DINESH KIRAN	(1012202006)
D. NIKHITHA	(1012202016)
P. HARI PRAKASH	(1012202009)
J. SRI CHAITHANYA	(1012202002)

Under the Esteemed Guidance of

Prof.C.NagaRaju,
Dean, Faculty of Engineering,
Head of the Department,
Computer Science and Engineering.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Y.S.R ENGINEERING COLLEGE OF YOGI VEMANA UNIVERSITY
Proddatur-516360, Y.S.R (Dt),ANDHRA PRADESH,2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Y.S.R ENGINEERING COLLEGE OF YOGI VEMANA UNIVERSITY
Proddatur-516360, Y.S.R (Dt)
ANDHRA PRADESH



CERTIFICATE

This is to certify that the project report entitled “DETECTION AND CLASSIFICATION OF FAKE NEWS USING DEEP LEARNING TECHNIQUE E-BERT(EXTENDED BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS)” is submitted by K. DINESH KIRAN, D. NIKITHA, P. HARI PRAKASH, J. SRI CHAITHANYA to the Y.S.R Engineering College of Yogi Vemana University, Proddatur, in partial fulfilment for the award of the degree of B. Tech in Computer Science and Engineering, is a Bonafide record of project work carried out by them under my supervision.

The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree or diploma.

Project Guide

Project Co-Ordinator

Head of the Department

Examiners:

Internal Examiner

External Examiner

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Y.S.R ENGINEERING COLLEGE OF YOGI VEMANA UNIVERSITY
Proddatur-516360, Y.S.R (Dt)
ANDHRA PRADESH



DECLARATION

We declare that this project report titled entitled “DETECTION AND CLASSIFICATION OF FAKE NEWS USING DEEP LEARNING TECHNIQUEE-BERT(EXTENDED BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS)” submitted in partial fulfilment of the degree of B. Tech in Computer Science and Engineering is a record of original work carried out by us under the Guidance of Prof.C.NagaRaju, The matter embodied in this project report has not been submitted by me for the award of any other degree or diploma.

Signature of the Guide

K. Dinesh Kiran	(1012202006)
D. Nikhitha	(1012202016)
P. Hari Prakash	(1012202009)
J. Sri Chaithanya	(1012202002)

ACKNOWLEDGMENTS

The satisfaction that accompanies the successful completion of any task would be incomplete without mention of the people, who made it possible, whose constant guidance and encouragement crowned our efforts with success. We take this opportunity to express my deepest gratitude and appreciation to all those who have helped us directly or indirectly towards the successful completion of this project.

It is a great pleasure to express a deep sense of gratitude and veneration to our guide **Prof. C. NAGARAJU**, Academic consultant in the Department of Computer Science and Engineering for her valuable guidance and thought-provoking discussion throughout the course of project work.

We extend our profound gratefulness to our **Prof. C. NAGARAJU**, Professor, Dean of Engineering and Head of the Computer Science and Engineering department for his encouragement and support throughout the project and also our project coordinators **Dr. R. PRADEEP KUMAR REDDY**, Associate Professor, and **Dr. S. KIRAN** Associate Professor, esteemed faculty at Department of Computer Science & Engineering, who helped us a lot in achieving success of the project.

We take this opportunity to offer gratefulness to our Honorable Vice-Chancellor **Prof. BELLAMKONDA RAJASEKHAR Garu**, our Vice-Principal **Prof. K. VENKATA RAMANAIAH Garu**, our Principal **Prof. G. JAYA CHANDRA REDDY Garu** for providing all sorts of environment during the project work.

We express our gratitude to **Dr. A. ASHOK KUMAR**, Associate Professor in the department of Science and Humanities, for support and putting up the extra time to reach our targets, which was greatly helpful in the progression and smoothness of the entire project work.

We express our thanks to all our college teaching and non-teaching staff members who encouraged and helped us in some way or other throughout the project work.

K. Dinesh Kiran	(1012202006)
D. Nikhitha	(1012202016)
P. Hari Prakash	(1012202009)
J. Sri Chaithanya	(1012202002)

LIST OF CONTENTS

CHAPTER NAME	PAGE NO
CHAPTER 1: INTRODUCTION	1–18
1.1 OVERVIEW	1
1.2 INTRODUCTION TO MACHINE LEARNING	1
1.2.1 Concept Of Machine Learning	2
1.2.2 Machine Learning Life Cycle	3
1.2.3 Classification Of Machine Learning	4
1.2.3.1 Supervised Learning	4
1.2.3.2 Unsupervised Learning	5
1.2.3.3 Reinforcement Learning	5
1.3 INTRODUCTION TO DEEP LEARNING	5
1.3.1 What is Deep Learning	5
1.3.2 Concept Of Deep Learning	6
1.3.3 Types Of Deep Learning Models	6
1.3.3.1 BERT (Bidirectional Encoder Representation From Transformers)	6
1.3.3.2 LSTM (Long Short–Term Memory)	8
1.3.3.3 GRU (Gated Recurrent Unit)	9
1.4 INTRODUCTION TO FAKE NEWS	10
1.4.1 Why Fake News Matters?	10
1.4.2 Key Characteristics Of Fake News	10
1.4.2.1 Intentional Creation	11
1.4.2.2 Heteromorphic Transmission	12
1.4.2.3 Controversial Reception	14
1.5 PROBLEM STATEMENT	16
1.6 NATURAL LANGUAGE PROCESSING	17
1.7 DATASET USAGE ACROSS MODELS	18

CHAPTER 2: LITERATURE SURVEY	19-27
CHAPTER 3: EXISTING METHOD	28-56
3.1 EXISTING MACHINE LEARNING MODELS FOR FAKE NEWS DETECTION	28
3.1.1 Support Vector Machine (SVM)	28
3.1.2 K-Nearest Neighbors (KNN)	32
3.1.3 Random Forest	34
3.1.4 Logistic Regression (LR)	36
3.1.5 Naïve Bayes (NB)	38
3.1.5.1 Naïve Bayes Theorem	40
3.1.6 Recurrent Neural Network (RNN)	41
CHAPTER 4:PROPOSED METHOD	57-111
4.1 INTRODUCTION TO THE PROPOSED METHOD	57
4.2 TYPES OF DEEP LEARNING MODELS	57
4.2.1 BERT (Bidirectional Encoder Representations From Transformers)	57
4.2.2 LSTM (Long ShortTerm Memory)	71
4.2.3 Gated Recurrent Unit (GRU)	81
4.3 PROPOSED METHODS	91
4.3.1 BERT+LSTM	91
4.3.2 BERT+GRU	98
4.4 RESULTS	109
CHAPTER 5: CONCLUSION AND FUTURE SCOPE	112
CHAPTER 6: BIBLIOGRAPHY	113-115

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
1.1	Process of the Traditional Logic Based Algorithm	2
1.2	Process of the Machine Learning Based Algorithm	3
1.3	AI-ML-Deep Learning Hierarchy	6
1.4	Bert Model Architecture	7
1.5	Long Short-Term Memory	8
1.6	Gated Recurrent Unit	9
1.7	Message Based Propagation	13
1.8	User Based Propagation	13
1.9	Hybrid Propagation	14
4.1	Bert Model Architecture	58
4.2	Forget Gate	72
4.3	Input Gate	72
4.4	Output Gate	73
4.5	Long Short-Term Memory	74
4.6	Gated Recurrent Unit	82
4.7	Bert With LSTM Architecture	92
4.8	Bert With GRU Architecture	99

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
3.1	Model Accuracy Values for Different Sample Sizes	46
3.2	Model Precision Values for Different Sample Sizes	48
3.3	Model Recall Values for Different Sample Sizes	50
3.4	Model F-1 Values for Different Sample Sizes	52
4.1	Performance of Proposed Models	109
4.2	Performance Metrics of EBERT (BERT+GRU) on Real-World Data	111
4.3	Performance Metrics of EBERT (BERT+LSTM) on Real-World Data	112

ABSTRACT

Abstract

With the rapid growth of Internet technology, sharing information has become faster and more convenient, but this has also led to the quick spread of fake news, which can seriously affect individuals, society, and even national stability. Because of this, detecting fake news has become an important research challenge. Fake news detection is mainly a classification task that verifies whether a given piece of news text is true or false. In this project, we focus on solving this problem using a deep learning approach based on BERT. By studying how fake news spreads, we identify three key characteristics: intentional creation, which explains why fake news is produced; heteromorphic transmission, which shows how it spreads in different forms across platforms; and controversial reception, which reflects the different opinions people have about the same news. Although traditional methods like Support Vector Machines (SVM) have been widely used, they often struggle with ambiguous language, idiomatic expressions, and complex context, resulting in lower accuracy. To overcome these issues, we propose E-BERT (Extended Bidirectional Encoder Representations from Transformers), which understands text by looking at both the left and right context of each word at the same time. This allows the model to capture deeper meaning, handle language nuances more effectively, and significantly improve the accuracy and reliability of fake news detection.

CHAPTER 1

INTRODUCTION

1.1 Overview

Fake news does not have a universally accepted definition. In this work, it is described as information that is intentionally created and verified to be false, with the clear purpose of misleading others. The main characteristics of fake news are false authenticity and a deliberate intent to deceive.

In contrast, rumours involve information whose authenticity has not yet been verified, and their intention is often unknown. Rumours may later turn out to be true or false, but at the time of circulation, their credibility remains uncertain.

Misinformation refers to false information that is shared without any harmful or misleading intent. In this case, the person spreading the information believes it to be true and does not aim to deceive others.

Fake news is considered a subset of the broader concept of disinformation, which includes all forms of information that are intentionally created and spread to deceive. Although this study mainly focuses on fake news detection, it also draws upon methods and insights from rumour and misinformation detection due to the strong similarities in their patterns and characteristics.

1.2 INTRODUCTION TO MACHINE LEARNING

Artificial Intelligence (AI) is one of the most influential technologies in the modern era and is widely used across various applications. Machine Learning (ML) is a major branch of AI that focuses on developing algorithms capable of learning from data rather than relying on explicitly programmed instructions. By adapting to the information provided, machine learning systems continuously improve their accuracy and performance over time.

Machine learning algorithms analyze available data to build models that can recognize patterns, make decisions, and handle new or unseen scenarios. The rapid growth and widespread adoption of machine learning can be mainly attributed to the following factors:

1.Huge amount of data – The availability of large-scale data from digital platforms provides rich information for training machine learning models.

2.Enhancement of computing power – Advances in hardware and parallel processing enable faster and more efficient model training.

3.Design of effective models with respect to data – Improved algorithms and model architectures allow better learning from complex and high-dimensional datasets.

Machine Learning is a subfield of Artificial Intelligence that significantly simplifies complex tasks by efficiently processing vast amounts of computational data. It is often described as a key technology behind the current commercial and technological revolution.

1.2.1 Concept of Machine Learning

Machine Learning is designed to learn from experience and improve its performance by adapting to a given hypothesis. At each stage of learning, the actions performed by the system generate new information, which is then used to refine its understanding and enhance future decisions. This continuous learning process enables the system to become more accurate and efficient over time.

Unlike conventional software development, the creation of machine learning algorithms does not rely on explicit programming. With industries generating massive volumes of data due to automated digitization, manually defining rules for every scenario has become impractical. Machine learning addresses this challenge by developing predictive models that automatically learn patterns from data and handle complex, time-consuming tasks.

Machine learning algorithms differ fundamentally from traditional algorithms. In a traditional approach, input data and predefined logic are provided to the algorithm, which then produces



Fig 1.1: Process of the traditional logic-based algorithm.

the desired output. In contrast, a machine learning approach provides input data along with an initial hypothesis, and the algorithm learns the underlying logic from the data itself. The conceptual difference between traditional algorithms and machine learning algorithms is illustrated in Figures 1 and 2.

Fig 1.2: Process of the machine learning based algorithm.

In traditional algorithms, the execution flow is predefined and remains fixed. In contrast, machine learning-based algorithms adapt their behaviour based on the learned hypothesis, and the resulting logic is then deployed into production.

1.2.2 Machine Learning Life Cycle

When an instruction is executed by the CPU, it follows a defined sequence of steps to produce an output. Similarly, machine learning algorithms operate through a systematic sequence of stages to generate meaningful results. This sequence of stages is known as the Machine Learning Life Cycle.

The key steps involved in the Machine Learning life cycle are as follows:

1. **Data Gathering** – Collecting relevant and reliable data from various sources.
2. **Data Pre-processing** – Cleaning the data and handling missing, noisy, or inconsistent values.
3. **Feature Engineering** – Selecting, extracting, and transforming features to improve model performance.

4. **Algorithm Selection and Training** – Choosing an appropriate machine learning algorithm and training it using the prepared data.
5. **Making Predictions** – Using the trained model to generate predictions or decisions on new, unseen data.

1.2.3 Classification of Machine Learning

Machine learning is basically classified into three types.

1. Supervised learning
2. Unsupervised learning
3. Reinforcement learning.

1.2.3.1 Supervised Learning

In this model's case, data to be considered for evaluation with a model must consist of a label.

In this type of learning algorithm, two kinds of tasks are performed.

1. Classification
2. Regression

Classification: -Classification is a task that categorizes the information depending on a categorical variable. A simple example of this is e-mail, which is already discussed. According to the classification, some important models are given below.

1. K-Nearest Neighbour (KNN)
2. Naïve Basie
3. Support Vector Machine
4. Decision Trees
5. Ensemble models

The ensemble models are multiple Machine Learning models gathered together to generate a good result.

Regression: -When there is a necessity to predict the continuous values in a given problem, regression is applied. It is a kind of model practiced from previous information and generates feature information.

In this, some important models are given below.

- 1.Logistic regression
- 2.Lasso regression
- 3.Ridge regression
- 4.Linear regression

1.2.3.2 Unsupervised Learning

Clustering is one of the models in Machine Learning which collects similar objects and makes them as a group. Effective supervised models are not available to perform clustering. To do such unsupervised models play a key role. The most widely used unsupervised models are K-Means, K Means, K-Medoids, Agglomerative clustering, and clustering with large dataset algorithms. Unsupervised algorithms are represented without any labelled data. Based on the similarity in the input information, information is gathered and represented as groups.

1.2.3.3 Reinforcement Learning

Reinforcement learning is one of the areas in the Machine Learning technique which is used to identify possible behaviour for a given model in a specific situation. It is differentiated from supervised learning in a way that supervised learning uses training data as an answer key for the model, whereas in reinforcement, there is no answer key, but the reinforcement agent decides how to execute the task in the absence of training data.

Key points: -

1. Input
2. Output
3. Training

In the case of input, it is to be referred to as the initial state from where the model will start. In the case of output variety of solutions may be generated for a given problem. In the training phase, depending on the input, the model will return an output. Here, the user needs

to decide whether to reward or punish the model. The best response is decided depending on the max reward.

1.3 INTRODUCTION TO DEEP LEARNING:

1.3.1 What is Deep Learning?

Deep learning is a subset of machine learning that is built on artificial neural network architectures. An artificial neural network (ANN) consists of multiple layers of interconnected units called neurons, which work together to analyze and learn patterns from input data.

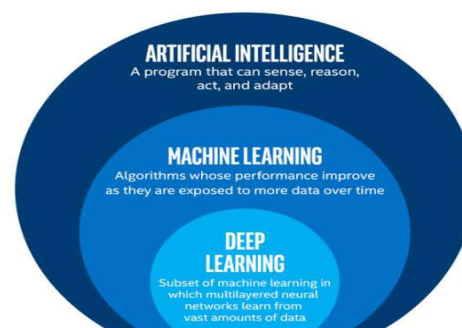
In a fully connected deep neural network, the structure includes an input layer followed by one or more hidden layers arranged sequentially. Each neuron receives inputs from neurons in the previous layer or directly from the input layer. The output generated by a neuron is passed as input to neurons in the next layer, and this process continues until the final layer produces the network's output.

Through these layered connections, the network applies a series of nonlinear transformations to the input data. This enables the model to capture complex relationships and learn meaningful representations from the data.

1.3.2 Concept of Deep Learning:

Deep learning is a part of machine learning that teaches computers to learn from data using structures called neural networks, which are inspired by how the human brain works. These networks are made of layers of connected nodes, called neurons, where each layer processes the data and passes it to the next.

By stacking many layers, deep learning models can automatically discover patterns and features in data, even very complex ones, without needing to be explicitly told what to look for. This ability makes deep learning especially powerful for tasks like recognizing images,



understanding speech, or analyzing text.

1.3.3 Types of Deep Learning Models:

1.3.3.1 BERT (Bidirectional Encoder Representations from Transformers)

BERT (Bidirectional Encoder Representations from Transformers) is a deep learning model designed for Natural Language Processing (NLP). Unlike traditional models that read text only from left-to-right or right-to-left, BERT reads text in both directions simultaneously. This bidirectional reading allows the model to understand the context of each word based on the surrounding words, which is essential for capturing meaning in sentences.

Architecture

BERT is built on the Transformer Encoder architecture and consists of multiple layers of self-attention mechanisms, which enable the model to focus on the most relevant words in a

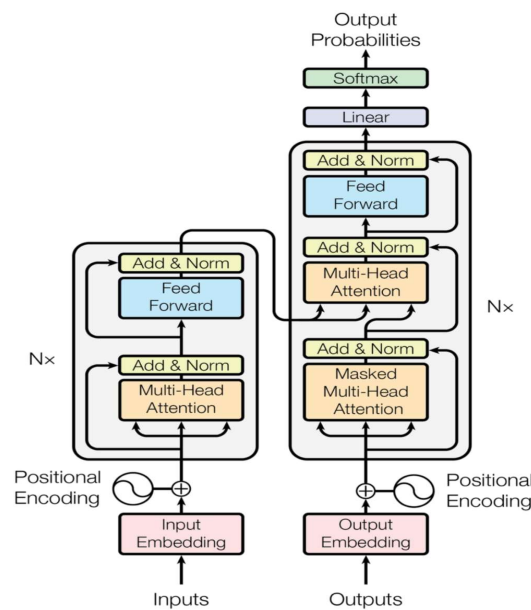


Fig1.4: Bert Model Architecture

ser

The Bert Transformer is a deep learning architecture that processes sequential data using attention mechanisms instead of recurrence, enabling parallel computation and efficient

learning. It consists of an encoder–decoder structure where input tokens are embedded and combined with positional encoding to preserve sequence order. Multi-head self-attention captures contextual relationships between tokens, while feed-forward networks refine representations, and a softmax layer generates the final output probabilities.

Input Components:

1. Token Embeddings – Represent each word in the input text.
2. Position Embeddings – Represent the position of each word in the sentence.
3. Segment Embeddings – Distinguish between two sentences when processing sentence pairs.

BERT generates contextualized vectors for each word, which capture the meaning of the word within its context. These vectors can then be used for various NLP tasks, such as:

- Text Classification
- Question Answering
- Named Entity Recognition (NER)

1.3.3.2 Long short-term Memory (LSTM):

LSTMs (Long Short-Term Memory) are a type of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data. They are particularly effective for processing sequences such as time series, text, and speech.

An LSTM uses a memory cell along with gates to control the flow of information. This mechanism allows the network to retain important information and discard irrelevant data, helping it avoid the vanishing gradient problem that traditional RNNs often face.

Due to their ability to handle sequential data effectively, LSTMs are widely used in applications like:

- Natural Language Processing (NLP)
- Speech Recognition

- Time Series Forecasting

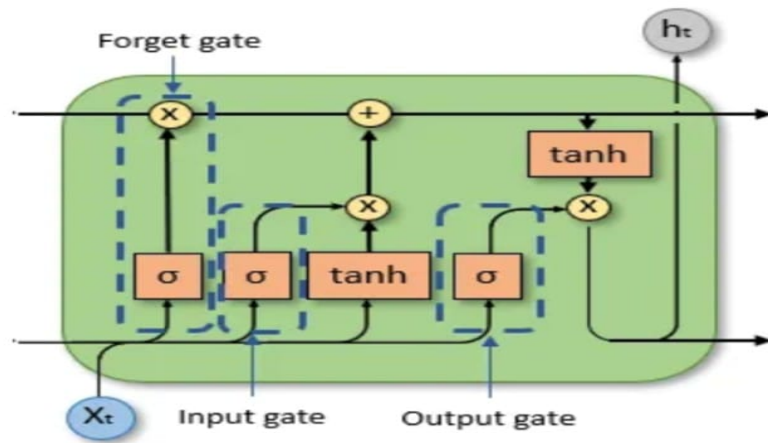


Fig 1.5: Long Short-term Memory

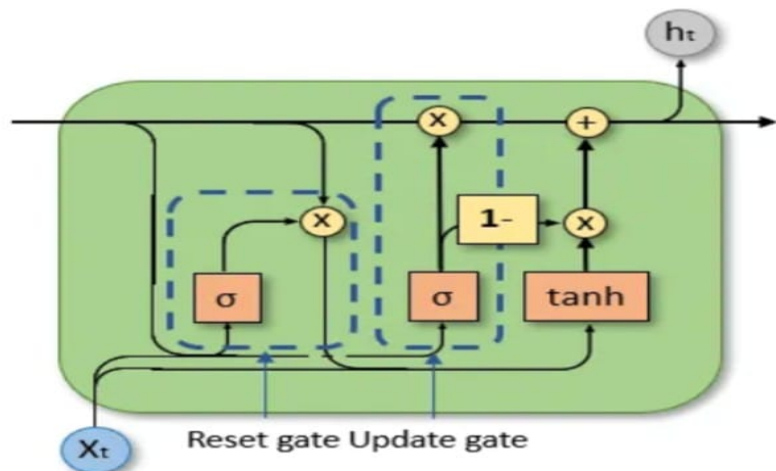
1.3.3.3 Gated Recurrent Unit (GRU)

GRUs (Gated Recurrent Units) are a simplified type of Recurrent Neural Network (RNN) designed to efficiently capture dependencies in sequential data. Like LSTMs, GRUs are effective for processing sequences such as time series, text, and speech.

A GRU uses a single memory unit along with two gates—the update gate and the reset gate—to control the flow of information. These gates decide how much past information should be retained and how much new information should be added. By reducing complexity compared to LSTMs, GRUs help mitigate the vanishing gradient problem while enabling faster training and lower computational cost.

Due to their efficiency and strong performance on sequential data, GRUs are widely used in applications such as:

- Natural Language Processing (NLP)



- Speech Recognition
- Time Series Forecasting

Fig 1.6: Gated Recurrent Unit

1.4 Introduction to Fake News

1.4.1 Why fake news matters?

The Internet and social media platforms have transformed the way information is shared, making communication faster and more widespread than ever before. News and updates can be delivered to a global audience within seconds, increasing awareness and connectivity. However, this speed also enables fake news to spread rapidly before proper verification can take place.

social panic

Fake news often creates fear, confusion, and unrest among the public. False information about emergencies, health issues, or security threats can cause panic reactions, leading people to make irrational decisions and lose trust in reliable sources. This disrupts social harmony and stability.

Political influence

Fake news has a powerful impact on political systems by manipulating public opinion and influencing democratic processes. Misleading or false political information can shape voter behaviour, increase polarization, and weaken trust in political institutions and governance.

Financial & Personal Harm

Fake news can result in serious financial losses through scams, fraudulent schemes, and market misinformation. It also causes personal harm by damaging reputations, affecting careers, and creating emotional stress for individuals who are falsely targeted or misrepresented.

1.4.2 Key Characteristics of Fake News

Fake news can be understood not only by its content but also by the way it is created and how it spreads across social media platforms. Unlike real news, which is produced through

verification and factual reporting, fake news follows unique patterns that reflect human intention and network behavior. By studying these patterns, we can identify important characteristics that distinguish fake news from genuine information. Two major intrinsic properties of fake news are intentional creation and heteromorphic transmission, which explain why fake news is produced and how it propagates through different types of users.

1.4.2.1 Intentional Creation

Fake news is deliberately created, unlike real news, which is based on truthful reporting of actual events. Its main purpose is to mislead the public, manipulate opinions, attract attention, or achieve political, social, or financial benefits. Fake news creators often design content to look realistic by using emotional language, sensational headlines, or misleading visual evidence, making it difficult for readers to distinguish it from genuine news. For example, an art image created in 2009 was later shared as real news during Hurricane Sandy to falsely claim that the Ronald McDonald statue had been flooded, which caused confusion and misinformation. This shows that fake news is not accidental; it is intentionally crafted to exploit trust and emotional responses.

Three Main Intentions

Mislead the Public

The goal here is to make people believe something that is not true. Fake news often appears realistic and credible so that readers do not easily suspect it. It uses subtle deceptive cues such as witness-like phrases (“I saw”, “I heard”), personal pronouns, and specific URL or mention patterns (@) to create a sense of authenticity. Special symbol features also help capture misleading writing styles, including excessive use of question marks and exclamation marks, similar text lengths, and repeated structural patterns. In addition, social influence cues such as mentions, comments, and reposts are considered because highly shared or frequently discussed content is more likely to be trusted by users. Contextual signals, like references to foreign or distant events that are hard to verify, also increase the possibility of misinformation.

Manipulate Opinions

In this case, fake news is created to shape how people think or feel about a topic, person, or event. This is usually achieved through emotionally charged language and extreme expressions. Sentiment features play an important role here, as fake news often shows strong positive or negative emotions compared to neutral and factual real news. Techniques such as sentiment analysis, sentiment lexicons, and linguistic patterns are used to identify emotional manipulation. Along with this, style features are very useful, because fake news often follows a distinctive writing style, especially in hyperpartisan content, which is more biased, aggressive, and persuasive than mainstream journalism. These stylistic differences help in detecting opinion manipulation.

Attract User Attention

Another major intention of fake news is to gain attention, clicks, and engagement. For this purpose, content is written in an eye-catching and sensational manner. It commonly includes exaggerated headlines, dramatic punctuation, strong emotions, and direct language to trigger curiosity and urgency. Typical features include URLs, mentions (@), frequent use of personal pronouns, excessive question and exclamation marks, and witness-like phrases. These elements make the news appear personal, urgent, and trustworthy, which encourages users to read, share, and react.

1.4.2.2 Heteromorphic Transmission

Heteromorphic transmission refers to the complex and diverse way fake news spreads through social networks. Unlike real news, which usually originates from trusted media sources or opinion leaders and then spreads to ordinary users, fake news often begins with low-impact or unknown users. Over time, it gains visibility and is amplified by influential users, such as opinion leaders or large social media accounts, which helps it reach a much wider audience. This multi-stage and indirect spreading pattern makes fake news harder to detect and control. Because it flows through different types of users and networks, fake news can blend into normal information streams, appear credible, and spread rapidly across platforms.

Types of Propagation Structures

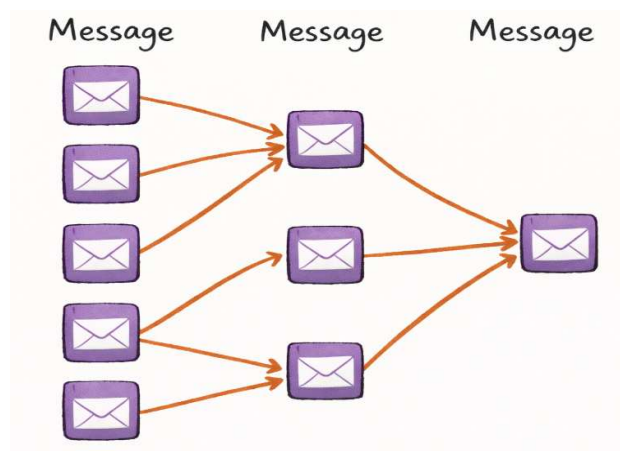
Fake news propagation structures are often more fragmented and multi-layered than those of real news. Instead of a simple source-to-user flow, fake news may pass through multiple types of accounts, including fake profiles, automated bots, and influential users, before

reaching the wider public. During this process, the message may be altered, shortened, or sensationalized, which increases its impact and makes detection more challenging. These diverse propagation structures are a key reason why fake news spreads faster and more widely than real news.

Three Modelling Strategies

Message-Based Propagation

This approach focuses on how the content of the message changes and spreads over time. It analyzes textual features, sentiment shifts, stylistic variations, and message mutations that occur as fake news is reposted or rephrased by different users. By tracking how the same information appears in different forms, this model helps identify patterns that are typical of



fake news

Fig 1.7: Message based propagation

User-Based Propagation

This strategy concentrates on the behaviour and characteristics of users involved in spreading fake news. It studies factors such as user credibility, influence level, posting frequency, and the presence of bots or spammers. Since fake news often begins with low-credibility users and later spreads through opinion leaders, modelling user roles is essential for understanding and detecting heteromorphic transmission

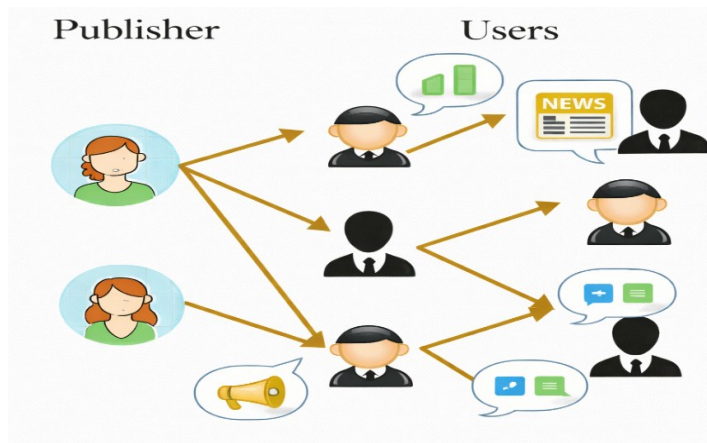


Fig1.8: User Based Propagation

Hybrid Propagation

The hybrid approach combines both message-based and user-based information. It simultaneously considers how the content evolves and who is spreading it. By integrating message features with user behaviour patterns, hybrid models provide a more complete and accurate representation of fake news propagation, making them highly effective for detecting complex and dynamic spreading behaviours.

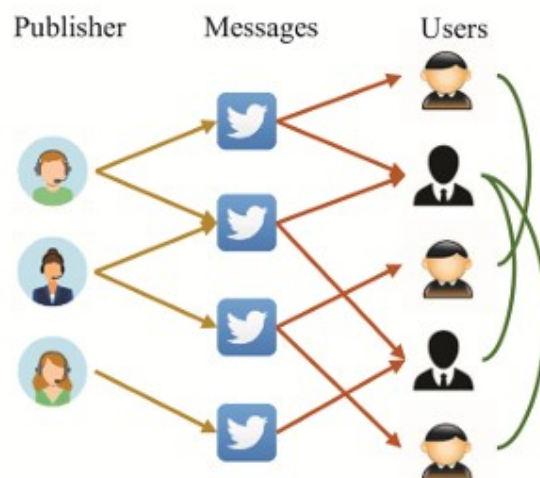


Fig 1.9: Hybrid Propagation

1.4.2.3 Controversial Reception

Fake news often receives controversial and mixed reactions from the public. Some users believe and accept the information as true, while others doubt its authenticity or completely reject it. This difference in perception arises because people have varying levels of awareness, knowledge, personal beliefs, and trust in information sources. On social media platforms, these opinions are commonly expressed through likes, dislikes, shares, comments, and debates. As a result, fake news creates divided viewpoints and discussions, which can further increase its visibility and spread. This controversial reception not only reflects the uncertainty surrounding fake news but also plays a role in amplifying its reach, as conflicting opinions encourage more engagement and interaction.

Users React Differently to Fake News

When fake news is posted, it typically generates three types of responses: support, denial, and questioning. Support comes from users who accept the news as true and help spread it further. Denial comes from users who recognize the news as false and try to correct or oppose it. Questioning comes from users who are unsure and seek clarification or evidence. The coexistence of these reactions shows that the content is controversial and possibly unreliable.

User Stance Types

Two Forms of Stance

Explicit Stance

Explicit stance is clearly expressed by users through their actions or language. It can be directly observed in:

- Comments that show agreement, rejection, or doubt
- Likes and dislikes
- Reposts that include personal opinions

In explicit stance-based detection, models use labeled user opinions to classify reactions into three main categories:

- **Support:** The user believes and promotes the news
- **Deny:** The user rejects and corrects the news
- **Question:** The user expresses uncertainty or asks for proof

These labels provide strong signals about the credibility of the news item.

Implicit Stance

Implicit stance is not directly stated but is hidden in user behavior and writing style. It can be inferred from subtle linguistic cues, such as sarcasm, tone, choice of words, or hesitation, as well as from user behavior patterns like limited sharing, delayed reactions, or selective engagement. Even without clearly saying “this is false” or “this is true,” users may reveal their stance through these indirect signals. Detecting implicit stance helps capture deeper and more realistic user opinions that are not always openly expressed.

So fake news is characterized by intentional creation, complex spreading patterns, and controversial user reactions. It is deliberately designed to mislead or manipulate, spreads irregularly through different users and networks, and generates mixed responses such as support, denial, and questioning. Understanding these three aspects provides a strong foundation for building effective fake news detection systems.

1.5 Problem Statement

Traditional BERT-based classifiers generate predictions for every input sample regardless of ambiguity or uncertainty in the data. These models do not provide any mechanism to measure the reliability of their predictions, and therefore all predictions are treated as equally trustworthy. This limitation makes conventional BERT classifiers unsuitable for applications that require reliable and interpretable decision-making without human supervision. In real-world environments such as fake news detection systems deployed on social media platforms, incorrect predictions made with high confidence can lead to widespread misinformation.

Similar risks exist in domains such as healthcare diagnosis, legal decision support, and financial fraud detection, where incorrect automated decisions may have severe consequences. Therefore, it becomes essential to design models that can recognize uncertain situations and indicate when human intervention is required.

In the proposed Confidence-Aware BERT with LSTM and GRU model enhances a standard BERT-based classifier by integrating an uncertainty estimation mechanism based on Monte Carlo Dropout. Instead of producing a single deterministic prediction, the model performs multiple stochastic forward passes during inference while dropout layers remain active. Each forward pass produces slightly different outputs due to randomly deactivated neurons, thereby generating a distribution of predictions for the same input sample. By analyzing the variation among these predictions, the model can estimate how confident it is about the final decision. This variation acts as an indicator of prediction uncertainty, enabling the model to distinguish between confident and uncertain predictions.

1.6 Natural Language Processing:

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, interpret, and process human language in a meaningful way. In this project, NLP is applied to analyze news articles and extract valuable information using the E-BERT method for fake news detection. By leveraging NLP techniques, the system can effectively process unstructured news text and convert it into structured, machine-readable data that the model can work with.

NLP helps in cleaning and preprocessing the text to remove noise and irrelevant information, making the data more suitable for analysis. It also breaks the text into smaller units called tokens, which allows the model to better understand the structure and meaning of sentences. By capturing the context and semantics of words, NLP enables the system to detect subtle cues, misleading statements, emotional tones, and biased language patterns that are often present in fake news.

Using these capabilities, NLP supports the accurate classification of news articles as fake or real. It improves the overall performance of the fake news detection system by ensuring that the model can focus on meaningful content, understand complex language structures, and

make reliable predictions. In this way, NLP forms the foundation of intelligent news analysis and plays a critical role in enhancing the effectiveness of the E-BERT method.

1.7 Dataset Usage Across Models

The dataset used in this study was obtained from Kaggle and consists of a text-based collection of **20,955 news articles**, labeled as *Real (0)* and *Fake (1)*, making it well-suited for supervised fake news classification tasks. The dataset contains full-length news content, representing unstructured long-form textual data across diverse domains such as politics, international affairs, and social issues.

Prior to model training, the dataset requires standard natural language preprocessing steps, including text cleaning, tokenization, and stopwords removal. For feature representation, multiple techniques were considered, such as TF-IDF, word embeddings, and contextual embeddings derived from BERT.

To ensure a fair and consistent evaluation, the same dataset was used across all models. Initially, it was applied to traditional deep learning sequence models such as LSTM and GRU, where textual data was transformed into numerical form using word embeddings. These models effectively captured sequential dependencies and contextual relationships within the text.

Furthermore, the dataset was utilized to train and evaluate advanced transformer-based architectures, including BERT, BERT + LSTM, and BERT + GRU. In these approaches, BERT served as a contextual feature extractor, generating rich bidirectional embeddings that capture deep semantic information. These embeddings were either directly used for classification or further processed through LSTM and GRU layers to enhance sequential learning and temporal dependency modeling.

Additionally, the dataset supports experimentation with both traditional machine learning models, such as Support Vector Machine (SVM) and Naïve Bayes, as well as deep learning approaches including RNN, LSTM, GRU, and BERT. By maintaining a consistent dataset across all experiments, the study ensures that performance comparisons remain objective and that observed differences are attributable to model architectures rather than data variability.

CHAPTER 2

LITERATURE SURVEY

2. Literature survey

In[1]Hisham Ahmed Mahmoud and Ibrahim M. Ibrahim (2025) proposed a study titled *Fake News Detection Using Natural Language Processing (NLP) and Machine Learning*. In this work, the researchers proposed an automated system to identify fake news by combining traditional statistical methods with advanced artificial intelligence techniques such as BERT to better understand context. They observed that fake news often uses emotional and repetitive language and emphasized that Explainable AI is essential for making detection decisions transparent to humans. The research explains how technology can detect fake news by identifying language patterns and shows that advanced AI methods are more accurate than older approaches because they understand meaning and context. The findings indicate that traditional methods are fast but limited, while AI-based methods provide better contextual understanding, and a hybrid approach that combines both speed and accuracy works best. However, the study highlights limitations such as high resource requirements, transparency issues, evolving fake news tactics, and scope constraints.

In[2]Ossama Embarak (2025) presented a study titled *Deep Learning for Fake News Detection: Analysing Facebook's Misinformation Networks*. The study proposes an Artificial Neural Network (ANN)-based fake news detection system designed for Facebook. The model analyzes news text, user sentiment from comments, and engagement patterns such as likes, shares, and comments. A color-coded truth-level indicator (Red-Fake, Orange-Suspicious, Green-True) is used to improve interpretability of results for users. The research findings show that the proposed ANN model achieved 89.2% accuracy, outperforming traditional models like Logistic Regression and SVM. User sentiment from comments is identified as a reliable indirect feedback signal for misinformation detection, and the study finds that fake news spreads faster and generates higher engagement than real news. The Red-Orange-Green truth-level indicator also improves transparency and user understanding of detection results. The limitations of this study include dataset bias due to imbalanced data, multilingual issues caused by translation errors in low-resource languages, latency challenges due to rapid misinformation spread, and generalization issues, as the model requires validation on other platforms such as Twitter and Reddit.

In[3]Suresh V. Reddy, Ashwini Wadekar, Bhavana Ghorpade, Sakshi Wagh, and Priya Sampate (2025) proposed a study titled *Fake News Detection Using Machine Learning and NLP*. In this work, the authors proposed a fake news detection system based on Machine Learning and Natural Language Processing techniques that classifies news articles as real or fake using text-based features. The approach emphasizes data preprocessing, TF-IDF feature extraction, and the use of traditional machine learning classifiers to achieve good accuracy with computational efficiency. The research findings show that text preprocessing techniques such as tokenization, stop-word removal, and normalization improve model performance, and TF-IDF effectively captures important textual features for fake news classification. Machine learning models like Logistic Regression and SVM provide reliable classification results, and proper preprocessing combined with traditional machine learning methods achieves satisfactory accuracy. However, the model considers only textual content, ignoring images, videos, and metadata, struggles with sarcasm, satire, and highly contextual news, is limited by dataset size and diversity affecting generalization, and does not explore deep learning or transformer-based models.

In[4]Bo Hu, Zhendong Mao, and Yongdong Zhang (2024) presented a study titled *An Overview of Fake News Detection: From a New Perspective*. The authors proposed a new perspective for fake news detection based on the complete diffusion process on social media and identified intrinsic characteristics such as intentional creation, heteromorphic transmission, and controversial reception. Detection methods were categorized into intentional feature-based, propagation-based, and stance-based approaches. The research findings indicate that fake news spreads deeper and wider than real news, user stance plays a crucial role in detection, deep learning models outperform traditional machine learning methods when multi-modal data is used, and combining content and social context improves early detection. The study also highlights several limitations, including the requirement for large labeled datasets, difficulty in early detection due to limited propagation data, poor cross-platform generalization, and a lack of explainability in deep learning models.

In[5]The study titled *A Hybrid Deep Learning Framework for Fake News Detection using LSTM-CGPNN and Metaheuristic Optimization* proposed a hybrid model that combines LSTM and CGPNN, optimized using MFWO. In this approach, LSTM captures contextual information while CGPNN performs probabilistic classification, and TF-IDF-based NLP

preprocessing is applied. The research findings show that the model achieved up to 98% accuracy and a 95% F1-score, outperforming CNN, transformer, and GNN models. It demonstrated a 3–8% improvement over state-of-the-art methods and produced consistent results across multiple datasets. However, the study has limitations such as high computational complexity requiring GPUs, performance drops in cross-dataset testing, restriction to English and political news, and a lack of explainability and robustness to adversarial content.

In[6]Faisal A. Alshuwaier and Fawaz A. Alsulaiman (2025) presented a study titled *Fake News Detection Using Machine Learning and Deep Learning Algorithms: A Comprehensive Review and Future Perspectives*. The paper provides a comprehensive review of machine learning and deep learning-based fake news detection methods, including datasets, features, evaluation metrics, and future research directions. The research findings indicate that deep learning models outperform traditional machine learning methods, transformer models such as BERT achieve over 95% accuracy, large datasets significantly improve performance, and hybrid ML–DL models provide robust results. The limitations discussed in the study include limited generalization due to small datasets, a strong focus on English-language studies, high computational cost, and the ongoing challenge of achieving real-time fake news detection.

In[7]The study titled *Detecting Fake News Using BERT Word Embedding, Attention Mechanism, Partition and Overlapping Text Techniques* proposed a BERT-based fake news detection model that uses attention mechanisms along with partition and overlapping text techniques. A hybrid CNN–BiLSTM–attention architecture was developed for effective classification. The research findings show that the model achieved nearly 99% accuracy on large datasets and outperformed traditional CNN and BiLSTM models. The use of partition and overlapping techniques further improved accuracy, and high precision, recall, and F1-score were achieved. However, the study requires large labeled datasets, is computationally expensive due to the use of BERT, experiences performance drops on imbalanced datasets, and ignores images, videos, and real-time detection aspects.

In[8]Enas Tariq Khudair et al. (2025) presented a study titled *Detecting Fake News Using Deep Learning Approaches*. The authors proposed a deep learning framework for Arabic fake news detection using DNN, CNN, LSTM, CNN–LSTM, and transformer-based models such as BERT and AraBERT. The research findings indicate that the DNN model achieved 78% accuracy, BERT achieved 99.1% accuracy, and AraBERT achieved the highest accuracy of

99.3%, demonstrating that deep learning models outperform traditional machine learning methods. The limitations of the study include the lack of large Arabic datasets, high computational cost, translation errors from English datasets, and the fact that real-time detection was not evaluated.

In[9]The study titled *Detection of Fake News Using Machine Learning and Natural Language Processing Algorithms* proposed an automated fake news detection system using machine learning, deep learning, and NLP techniques. The system applies text preprocessing methods such as regex, tokenization, stopwords removal, and lemmatization, followed by TF-IDF feature extraction and classification using machine learning models and deep learning models like LSTM and BERT. The research findings show that the Decision Tree model achieved 90% accuracy, outperforming traditional machine learning models, while LSTM achieved 95% accuracy with improved contextual understanding. NLP-based BERT achieved the highest accuracy of 98%, and overall, deep learning and transformer models outperformed traditional machine learning approaches. However, the study relies mainly on textual data and ignores images and videos, involves high computational cost for LSTM and BERT models, requires large labeled datasets for optimal performance, and does not implement real-time fake news detection.

In[10]The study titled *Enhancing Fake News Detection via Stance Analysis: Leveraging Advanced NLP Techniques and Machine Learning Models* proposed a stance-based fake news detection framework by analyzing the relationship between news headlines and body text. The approach integrates NLP preprocessing, TF-IDF, named entity recognition, sentiment analysis, and semantic similarity, using machine learning models and transformer-based DistilBERT. The research findings indicate that transformer-based DistilBERT achieved the best performance with approximately 90% test accuracy, stance analysis improves the detection of nuanced misinformation, context-aware models outperform traditional machine learning classifiers, and combining linguistic and semantic features enhances robustness. The limitations include difficulty in distinguishing closely related stance classes, the requirement of large annotated stance datasets, high computational cost of transformer models, and ongoing challenges in real-time deployment.

In[11]Shahzad Bhatti et al. (2023) presented a study titled *Deep Learning with Graph Convolutional Networks: An Overview*. This paper provides a comprehensive overview of Graph Convolutional Networks (GCNs) in deep learning and explains how GCNs effectively

model graph-structured data by learning node and edge representations. The study discusses GCN architectures, training methods, and different variations used across applications, highlighting their use in domains such as fake news detection, social networks, and recommendation systems. The research findings indicate that GCNs are highly effective in capturing relationships and dependencies in graph data and that GCN-based models outperform traditional deep learning models on graph-related tasks. The study also shows that GCNs improve performance in applications like fake news detection by leveraging social context and that combining GCNs with other deep learning models enhances classification accuracy. However, the paper notes several limitations, including high computational complexity on large graphs, performance degradation due to over-smoothing in deep GCN layers, the requirement for high-quality graph data that is not always available, and scalability challenges for real-time applications.

In[12] Mohammad Q. Alnabhan and Paula Branco (2024) proposed a study titled *Fake News Detection Using Deep Learning: A Systematic Literature Review*. The paper presents a systematic literature review of fake news detection techniques based on deep learning and analyzes various models such as CNN, RNN, LSTM, GRU, and transformer-based approaches. It reviews publicly available datasets used for fake news detection and examines the role of transfer learning and class imbalance handling techniques in improving detection accuracy. The research findings show that deep learning models perform better than traditional machine learning methods for fake news detection, with LSTM, CNN, and BERT-based models achieving higher accuracy than other approaches. The study highlights that transfer learning, especially using BERT, significantly improves fake news detection performance and that handling class imbalance enhances precision and recall in fake news datasets. The limitations discussed include the lack of large, high-quality labeled datasets, overfitting and poor generalization to new datasets, high computational costs associated with deep learning and transformer models, and limited research on cross-domain and multilingual fake news detection.

In[13] Shweta Kumari and Maheshwari Prasad Singh (2024) proposed a study titled *A Deep Learning Multimodal Framework for Fake News Detection*. The paper proposes a multimodal deep learning framework that combines textual and visual features to detect fake news. It uses Natural Language Processing techniques for text analysis and Convolutional Neural

Networks for image feature extraction, and the model is trained and evaluated on benchmark fake news datasets. The research findings show that the multimodal approach outperforms unimodal models that rely only on text or images. The proposed framework achieves higher accuracy, precision, and recall in fake news classification, and combining text and image information helps in identifying misleading and manipulated content more effectively. Experimental results also demonstrate the robustness of the model across different datasets. However, the model requires large labeled datasets that are difficult to collect, involves high computational cost due to deep learning and multimodal processing, performance may degrade when images are missing or irrelevant, and the framework is less effective for real-time fake news detection.

In[14]Nabeel Raza, Said Jadid Abdulkadir, Yawar Abbas Abid, Sami S. Albouq, Ayed Alwadain, Asad Ur Rehman, Ebrahim Hamid Sumiea, and Muhammad Farhan (2024) presented a study titled *Enhancing Fake News Detection with Transformer-Based Deep Learning: A Multidisciplinary Approach*. The paper proposes a transformer-based deep learning model for fake news detection using pre-trained transformer models such as BERT to capture contextual meaning from news text. The approach integrates linguistic, semantic, and contextual features for better classification and applies a multidisciplinary framework by combining computer science and social science insights. The research findings indicate that the transformer-based model achieves higher accuracy compared to traditional machine learning methods and performs well in detecting complex and misleading fake news content. Experimental results show improved precision and recall on benchmark datasets, and the approach is effective across different types of news domains. The limitations include the requirement of large labeled datasets for effective training, high computational cost and memory usage due to the transformer architecture, reduced performance for low-resource languages, and limited suitability for real-time fake news detection due to processing time.

In[15]Muhammad Abdullah, Zan Hongying, Arifa Javed, Orken Mamyrbayev, Fabio Caraffini, and Hassan Eshkiki (2024) proposed a study titled “A Joint Learning Framework for Fake News Detection.” The paper introduces a joint learning framework that combines news content features with social context features to improve fake news detection. The proposed approach jointly learns representations from textual news content and user interaction data using deep learning models. By integrating semantic features from text and propagation patterns from social behavior, the framework captures complex relationships involved in fake news dissemination. The goal of the model is to improve detection accuracy

by learning multiple feature types simultaneously rather than relying on a single feature source. The research findings show that the joint learning approach achieves better accuracy compared to models that use only content-based or social-based features independently. The model effectively detects fake news by analyzing both content semantics and user behavior patterns. Experimental results demonstrate improved precision and recall on benchmark datasets, and the framework performs well in identifying early-stage fake news. However, the model heavily depends on social network data, which may not always be available. It also suffers from high computational complexity due to the joint learning architecture. Performance may degrade when user interaction data is incomplete or noisy, and the framework is less effective for cross-platform fake news detection.

In[16] Nabeel Raza et al. (2024) proposed a study titled “Enhancing Fake News Detection by Multi-Feature Classification.” This paper presents a multi-feature fake news detection framework that integrates textual, linguistic, and semantic features to improve classification performance. The approach combines machine learning and deep learning models, supported by NLP techniques such as text preprocessing and feature extraction. Transformer-based and ensemble models are used to enhance fake news classification across multiple datasets. The main objective of the framework is to improve robustness and accuracy by leveraging diverse feature representations. The research findings indicate that the proposed model achieves higher accuracy compared to traditional single-feature approaches. Combining multiple features significantly improves fake news classification performance. Transformer-based and ensemble models outperform basic machine learning models, and the system demonstrates good generalization ability on benchmark fake news datasets. Despite its effectiveness, the model requires large labeled datasets, which are time-consuming to collect. Performance may decrease when applied to real-time or multilingual news data. The use of deep learning models leads to high computational cost, and the framework may struggle to adapt to new and evolving fake news patterns.

In[17] Shivam Bansal, Sakshi Gupta, and Ankit Kumar (2024) proposed a study titled “*Text Data Augmentation Techniques for Word Embeddings in Fake News Classification*.” The paper proposes the use of text data augmentation techniques to improve fake news classification performance. Different augmentation methods such as synonym replacement, word insertion, and word swapping are applied. Word embedding models like Word2Vec and GloVe are used for feature representation. The main objective of the approach is to reduce data scarcity and enhance model robustness. The research findings show that data

augmentation significantly improved classification accuracy compared to non-augmented data. Models trained with augmented data demonstrated better generalization on unseen news samples. Word embedding-based classifiers performed better with balanced and enriched datasets, and the approach helped reduce overfitting in fake news detection models. However, some augmentation techniques may introduce semantic noise into the dataset. The effectiveness of augmentation depends on the quality of the original dataset, and results may vary across different languages and domains. Additionally, deep learning models still require high computational resources.

In[18]Ahmed H. Qureshi, Mohammad H. Alshahrani, and Fahad Alqahtani (2024) proposed a study titled “*WELFake: Word Embedding over Linguistic Features for Fake News Detection.*” The paper introduces the WELFake framework, which combines word embeddings with linguistic features for fake news detection. Linguistic features such as syntax, grammar, and writing style are integrated with semantic representations. Pre-trained word embedding models are used to capture the contextual meaning of news text. The approach aims to improve fake news detection accuracy by leveraging both semantic and linguistic cues. The research findings indicate that the WELFake model achieved better accuracy compared to traditional text-based methods. Combining linguistic features with word embeddings improved classification performance, and the approach showed robustness across different fake news datasets. The results confirmed that linguistic patterns are important indicators of fake news. However, the model performance depends on the quality of extracted linguistic features. The framework involves high computational complexity due to feature extraction and embedding processing. It may not generalize well to multilingual or low-resource languages and requires careful preprocessing to avoid noise in linguistic data.

In[19]Ammar Oad, Hamza Farooq, Amna Zafar, Beenish Ayesha Akram, Ruogu Zhou, and Feng Dong (2024) proposed a study titled “*Fake News Classification Methodology with Enhanced BERT.*” The authors proposed an Enhanced BERT-based fake news classification model by adding custom linear, dropout, and activation layers on top of the BERT-base model. A regional dataset named PolitiTweet was developed, focusing on political tweets with cultural and regional context. Data augmentation techniques were applied to handle class imbalance and improve model generalization. The Enhanced BERT model was compared with state-of-the-art models such as XLNet, RoBERTa, GPT-2, GPT-3, and Adversarial Networks. The research findings show that the Enhanced BERT model outperformed baseline BERT and other deep learning models on both datasets. It achieved

98.23% accuracy on the PolitiTweet dataset and approximately 87% accuracy on the BuzzFeed dataset. The model showed higher precision, recall, and F1-score, indicating strong reliability in detecting fake news. Attention-based explainability helped identify important linguistic features, thereby improving model interpretability. However, the model is computationally expensive and requires high-end GPU resources for training. The evaluation was limited to English-language datasets, reducing multilingual applicability. Performance may drop when applied to real-time or rapidly evolving fake news content, and the model mainly focuses on text-based data while ignoring images, videos, and other multimodal signals.

In[20]Jawaher Alghamdi, Yuqing Lin, and Suhuai Luo (2025) proposed a study titled “*ABERT: Adapting BERT Model for Efficient Detection of Human and AI-Generated Fake News.*” The authors proposed ABERT, a parameter-efficient fake news detection model based on BERT. Instead of fully fine-tuning BERT, the original BERT model is frozen and lightweight adapter layers are added. The model is designed to detect both human-written and AI-generated fake news. ABERT aims to maintain high accuracy while reducing computational cost and the number of trainable parameters. The research findings indicate that ABERT reduces the number of trainable parameters by about 67.7% compared to fully fine-tuned BERT. The model achieves performance comparable to or better than state-of-the-art models on FakeNewsNet and AI-generated datasets. ABERT performs well even without using social media context, relying only on news content, and is suitable for large-scale and resource-limited environments. However, the study focuses only on textual content and does not use social or user behaviour features. Experiments were conducted using BERT-base and not BERT-large due to hardware limitations. The maximum input length was limited to 128 tokens, which may miss long-context information, and the adapter structure is simple, where more complex adapters might improve accuracy but increase parameters.

CHAPTER 3

EXISTING METHOD

3.1 Existing Machine Learning Models for Fake News Detection

3.1.1 SUPPORT VECTOR MACHINE (SVM)

Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression problems. It is particularly effective for high-dimensional data, which makes it highly suitable for text-based applications such as fake news detection. In text classification, the number of features can be extremely large because each unique word or phrase may act as a feature. SVM handles such high-dimensional feature spaces efficiently and provides strong generalization performance. In fake news detection, SVM is used to classify news articles into **fake** or **real** categories based on textual features extracted from the content. The key objective of SVM is to find an optimal decision boundary, called a **hyperplane**, that separates the two classes with maximum margin.

Intuition Behind SVM

The intuition behind SVM is to separate fake news and real news using a boundary such that the distance between this boundary and the closest data points from both classes is as large as possible. These closest data points are known as **support vectors**. By maximizing this margin, SVM reduces overfitting and improves prediction accuracy on unseen news articles.

Mathematical Representations

Decision Function

$$f(x) = w \cdot x + b \quad (1)$$

Where:

- $f(x)$ – Decision value used to classify the input
- w – Weight vector defining the orientation of the hyperplane
- x – Feature vector representing a news article (e.g., TF-IDF values)
- b – Bias term that shifts the hyperplane

Optimization Objective

$$\min \frac{1}{2} \|w\|^2 \quad (2)$$

Where:

- $\|w\|$ – Euclidean norm (magnitude) of the weight vector
- Minimizing this term maximizes the margin between classes

This formulation ensures maximum separation between fake and real news classes.

Constraint Condition

$$y_i(w \cdot x_i + b) \geq 1 \quad (3)$$

Where:

- y_i – Class label of the i -th sample (-1 = Real, $+1$ = Fake)
- x_i – Feature vector of the i -th training sample

RBF Kernel Function

$$K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2} \quad (4)$$

Where:

- $K(x_i, x_j)$ – Kernel similarity between two samples
- γ – Kernel parameter controlling influence of data points
- $\|x_i - x_j\|$ – Distance between feature vectors

Working Mechanism and Role of Support Vector Machine (SVM) in Fake News

Detection and Classification

Support Vector Machine (SVM) plays a crucial and central role in fake news detection by functioning as a powerful supervised machine learning classifier that distinguishes between fake and real news articles based on their textual characteristics. Fake news detection is a binary text classification problem in which news content is analyzed and assigned to one of two classes: fake or real. Since textual data is inherently high-dimensional and sparse after feature extraction, SVM is particularly well suited for this task because it performs efficiently

when the number of features is very large compared to the number of training samples. In a typical fake news detection system, news articles are first collected in a labelled dataset and then pre-processed through steps such as lowercasing, removal of punctuation, numbers, special characters, and stop words, along with stemming or lemmatization. These preprocessing steps help normalize the text, reduce noise, and improve the model's ability to generalize.

After preprocessing, SVM operates on numerical representations of news articles rather than raw text. Feature extraction techniques such as Term Frequency–Inverse Document Frequency (TF-IDF) are commonly used to convert each article into a high-dimensional feature vector that captures the importance of words within the document relative to the entire dataset. The primary role of SVM at this stage is to learn an optimal decision boundary, known as a hyperplane, that best separates fake news articles from real news articles in the feature space. SVM achieves this by maximizing the margin between the two classes, where the margin represents the distance between the closest data points of each class. These closest points, called support vectors, are critical because they define the position of the hyperplane. By focusing on support vectors and the maximum-margin principle, SVM ensures robust and reliable classification even in complex, high-dimensional text spaces.

In real-world fake news datasets, perfect separation between fake and real news is rarely possible due to noisy data, overlapping vocabulary, and ambiguous language. To address this, SVM uses a soft-margin approach that allows limited misclassifications while still maintaining a large margin, thereby improving generalization to unseen data. Linear kernels are most commonly used in fake news detection because text data represented through TF-IDF is often linearly separable and linear SVMs are computationally efficient and scalable. During training, SVM learns the optimal weight vector and bias that define the decision function, and during prediction, it classifies new articles as fake or real based on their position relative to the learned hyperplane. The effectiveness of SVM is evaluated using metrics such as accuracy, precision, recall, and F1-score, where SVM often demonstrates high performance, especially in imbalanced datasets. Overall, SVM provides an accurate, efficient, and robust framework for fake news detection by learning discriminative textual patterns and making reliable classification decisions.

Advantages of SVM in Fake News Detection

1. **Effective handling of high-dimensional text data:** SVM performs exceptionally well with high-dimensional and sparse feature spaces created by text vectorization techniques such as TF-IDF. This makes it highly suitable for fake news detection, where each news article may be represented by thousands of features.
2. **Strong generalization and reduced overfitting:** By maximizing the margin between fake and real news classes, SVM achieves better generalization on unseen data. This property helps the model maintain high classification accuracy even when dealing with limited or noisy training datasets.
3. **Robust performance on noisy and overlapping data:** SVM uses a soft-margin approach and focuses on support vectors, allowing it to handle ambiguity and overlapping vocabulary between fake and real news effectively. This results in reliable classification performance in real-world fake news scenarios.

Disadvantages of SVM in Fake News Detection

1. **High computational cost for large datasets:** Training SVM models can be time-consuming and memory-intensive, especially when handling very large datasets or using non-linear kernels, which limits scalability in some applications.
2. **Sensitive to hyperparameter selection:** The performance of SVM depends heavily on choosing appropriate parameters such as the regularization constant and kernel type. Improper tuning can lead to poor classification results.
3. **Limited interpretability:** SVM does not provide easily understandable explanations for its predictions, making it difficult to interpret why a particular news article is classified as fake or real, which can be a drawback in systems requiring transparency.

3.1.2 K-NEAREST NEIGHBORS (KNN)

Introduction

K-Nearest Neighbors (KNN) is a simple and widely used supervised machine learning algorithm primarily employed for classification and regression tasks. It is a non-parametric and instance-based learning method, meaning it does not make assumptions about the

underlying data distribution and does not build an explicit training model. Instead, KNN stores the entire training dataset and makes predictions by measuring the similarity between a new data point and the existing labeled data points. Classification is performed by identifying the k nearest neighbors based on a chosen distance metric, such as Euclidean, Manhattan, or cosine distance, and assigning the class label through majority voting. Due to its simplicity, ease of implementation, and intuitive decision-making process, KNN is commonly used as a baseline algorithm in machine learning applications.

Intuition Behind KNN

The intuition behind K-Nearest Neighbors (KNN) is based on the idea that data points with similar characteristics tend to belong to the same category. When a new data point is introduced, the algorithm compares it with existing labeled data and identifies the k closest points using a distance measure. These nearby points represent the most similar examples in the dataset. The class of the new data point is then determined by the majority class among its nearest neighbors.

Mathematical Formulation

K-NEAREST NEIGHBORS (KNN)

(a) Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (5)$$

Where:

- $d(x, y)$ – Distance between test and training samples
- x_i – Feature value of test article
- y_i – Feature value of training article
- n – Total number of features

Cosine Similarity

$$\text{Cosine}(x, y) = \frac{x \cdot y}{|x||y|} \quad (6)$$

Where:

- $x \cdot y$ – Dot product of feature vectors

- $|x|, |y|$ – Magnitudes (lengths) of vectors
- Value close to 1 indicates high similarity

Application and Functioning of K-Nearest Neighbors (KNN) in Fake News

Classification:

K-Nearest Neighbors (KNN) plays an important role in fake news detection by classifying news articles based on their similarity to previously labeled articles. Fake news detection is a binary classification problem in which news content is categorized as fake or real. In the KNN-based detection process, news articles are first converted into numerical feature representations using techniques such as Term Frequency–Inverse Document Frequency (TF-IDF). These feature vectors capture the importance of words within each article and allow similarity comparisons between articles. KNN does not build an explicit predictive model; instead, it relies on stored training data and evaluates similarity at the time of prediction.

The working process of KNN in fake news detection begins by computing the similarity or distance between a new (test) news article and all articles in the training dataset using distance measures such as cosine similarity or Euclidean distance. Based on these similarity scores, the algorithm selects the k nearest neighbors—those articles that are most similar to the test article. The class labels of these neighbors are then examined, and a majority voting mechanism is applied to determine the final class of the test article. If most of the nearest neighbors are labeled as fake, the article is classified as fake; otherwise, it is classified as real.

The role of KNN in fake news detection is particularly effective because fake news articles often occur in clusters that share similar wording, tone, and structural patterns. KNN captures these patterns by comparing new articles directly with known fake and real news samples. By relying on local similarity rather than a global decision boundary, KNN can adapt to variations in language and evolving writing styles. This instance-based learning approach makes KNN a useful baseline and comparative method in fake news detection systems, especially when interpretability and simplicity are important.

Advantages of K-Nearest Neighbors (KNN)

- Simple and intuitive algorithm that is easy to understand and implement
- No explicit training phase, as the model directly uses stored data for prediction
- Effective for capturing local patterns and similarity-based relationships in data

Limitations of K-Nearest Neighbors (KNN)

- Slow prediction time for large datasets due to distance computation with all samples
- High memory usage since the entire training dataset must be stored
- Performance is sensitive to the choice of k value and distance metric

3.1.3 RANDOM FOREST

Introduction

Random Forest is a widely used ensemble machine learning algorithm that builds multiple decision trees and combines their predictions to produce a more accurate and stable result. Instead of relying on a single decision tree, Random Forest creates a collection of trees using different subsets of the training data and features, which helps reduce overfitting and improve generalization. Each tree makes its own prediction, and the final output is determined by majority voting in classification tasks or averaging in regression tasks. Due to its robustness, ability to handle complex data, and resistance to noise, Random Forest is commonly applied in a wide range of machine learning problems.

Intuition Behind Random Forest

The intuition behind Random Forest is based on the idea that **a group of diverse models can make better and more reliable predictions than a single model**. Instead of depending on one decision tree, Random Forest builds many trees using different random subsets of data and features, so each tree learns different patterns. While individual trees may make mistakes, combining their predictions through voting or averaging helps cancel out errors. This collective decision-making process results in improved accuracy, reduced overfitting, and more stable predictions.

Mathematical Concepts

Gini Index

$$Gini = 1 - \sum p_i^2 \quad (7)$$

Where:

- p_i – Probability of class i at a node

- Gini measures impurity of a decision tree node

Entropy

$$Entropy = -\sum p_i \log_2 p_i \quad (8)$$

Where:

- p_i – Probability of class i
- Entropy measures randomness or uncertainty in the data

Working Mechanism and Role of Random Forest in Fake News Detection and Classification

Random Forest plays an important role in fake news detection by serving as a robust ensemble classifier that combines the predictions of multiple decision trees to accurately distinguish between fake and real news articles. Fake news detection is a binary classification problem involving complex, high-dimensional textual data that often contains noise, misleading language, and overlapping patterns. In the working process of Random Forest, multiple subsets of the training data are created through random sampling with replacement, and for each subset, an independent decision tree is constructed. Each tree learns different patterns from the data, allowing the model to capture a wide variety of linguistic and structural features present in news articles.

During the classification process, each decision tree in the Random Forest independently analyses a news article and predicts whether it is fake or real based on the learned decision rules. Because the trees are trained on different data samples and feature subsets, they produce diverse predictions. The final classification decision is then made using a majority voting mechanism, where the class predicted by most trees is selected as the output. This ensemble approach reduces the influence of individual tree errors and leads to more stable and reliable predictions compared to using a single decision tree.

The role of Random Forest in fake news detection is particularly significant because fake news data is often noisy and contains complex, misleading patterns. Random Forest effectively handles such data by reducing variance and minimizing overfitting through ensemble learning. By aggregating multiple decision trees, the model captures diverse indicators of fake news, such as exaggerated language, unusual phrasing, or inconsistent

structure. Overall, Random Forest provides a powerful, accurate, and scalable framework for fake news detection and classification, making it well suited for real-world applications.

Advantages of Random Forest

- High accuracy due to ensemble learning and majority voting
- Robust to noise and overfitting by averaging multiple decision trees
- Handles large feature sets and complex data effectively

Limitations of Random Forest

- Less interpretable compared to a single decision tree
- Computationally expensive in terms of training and memory usage
- Slower prediction time when a large number of trees are used

3.1.4 LOGISTIC REGRESSION (LR)

Introduction

Logistic Regression is a widely used supervised machine learning algorithm designed for classification tasks, especially binary classification problems. Unlike linear regression, it predicts the probability of a data point belonging to a particular class by applying a logistic (sigmoid) function to a linear combination of input features. The output probability is then mapped to a class label based on a threshold value, typically 0.5. Due to its simplicity, interpretability, and efficiency, Logistic Regression is commonly used as a baseline model in many machine learning and data analysis applications.

Mathematical Formulation

Linear Model

$$z = w \cdot x + b \quad (9)$$

Where:

- z – Linear combination score
- w – Weight vector
- x – Feature vector

- b – Bias term

Sigmoid Function

$$\sigma(z) = \frac{1}{1+e^{-z}} \quad (10)$$

Where:

- $\sigma(z)$ – Probability that the article is fake
- e – Euler’s number (≈ 2.718)

Decision Rule

$$\sigma(z) \geq 0.5 \Rightarrow \text{Fake News}$$

$$\sigma(z) < 0.5 \Rightarrow \text{Real News}$$

Where:

- 0.5 – Classification threshold

Working Mechanism and Role of Logistic Regression in Fake News Detection and Classification

Logistic Regression plays an important role in fake news detection by serving as an efficient and interpretable classification algorithm for distinguishing between fake and real news articles. In the working process, news articles are first preprocessed and converted into numerical feature representations using techniques such as Term Frequency–Inverse Document Frequency (TF-IDF). These feature vectors capture the importance of words within each article and act as input to the Logistic Regression model. The algorithm then computes a weighted linear combination of these features and applies the sigmoid function to estimate the probability that a given news article belongs to the fake or real class.

During classification, the probability generated by the sigmoid function is compared against a predefined threshold, typically 0.5, to assign the final class label. If the predicted probability exceeds the threshold, the article is classified as fake; otherwise, it is classified as real. The role of Logistic Regression in fake news detection is particularly valuable due to its interpretability, as it assigns meaningful weights to individual words or features. These weights help identify terms that are strongly associated with fake news, such as exaggerated

or sensational phrases. Overall, Logistic Regression provides a simple, transparent, and effective framework for fake news detection and is often used as a baseline or comparative model in classification systems.

Advantages of Logistic Regression

- Simple and highly interpretable model with clear feature importance
- Fast and computationally efficient for training and prediction
- Works well with high-dimensional sparse data such as TF-IDF features

Limitations of Logistic Regression

- Assumes a linear decision boundary, limiting its ability to model complex patterns
- Performance decreases when classes are not linearly separable
- Sensitive to outliers and multicollinearity in features

3.1.5 NAIVE BAYES (NB)

Introduction

Naive Bayes is a simple and widely used probabilistic machine learning algorithm based on Bayes' Theorem. It works by calculating the probability of a data point belonging to a particular class based on prior probabilities and observed features. The algorithm assumes conditional independence among features, which simplifies computation and makes it highly efficient. Despite this simplifying assumption, Naive Bayes often provides reliable and fast classification performance, especially when dealing with large datasets.

Mathematical Formulation

Bayes' Theorem

$$P(C | X) = \frac{P(X|C)P(C)}{P(X)} \quad (11)$$

Where:

- $P(C | X)$ – Posterior probability of class C given features X
- C – Class label (Fake or Real)
- X – Feature vector (set of words)

- $P(X | C)$ – Likelihood of features given class
- $P(C)$ – Prior probability of class
- $P(X)$ – Evidence (normalizing constant)

Independence Assumption

$$P(X | C) = \prod_{i=1}^n P(x_i | C) \quad (12)$$

Where:

- x_i – Individual word or feature
- n – Number of features
- Assumes features contribute independently

Prior Probability

$$P(C) = \frac{\text{documents in class } C}{\text{total documents}}$$

Where:

- $P(C)$ – Probability of class C

Multinomial Naive Bayes (Text Classification)

$$P(w | C) = \frac{\text{count}(w, C) + 1}{\sum \text{count}(w, C) + V}$$

Where:

- w – Word in the vocabulary
- $\text{count}(w, C)$ – Frequency of word w in class C
- V – Vocabulary size
- $+1$ – Laplace smoothing to avoid zero probability

3.1.5.1 Naïve Bayes' Theorem

Naive Bayes is built on Bayes' Theorem, which provides a mathematical way to calculate the probability of a class given a set of observed features. The posterior probability ($P(C | X)$) is computed using the likelihood ($P(X | C)$), the prior probability ($P(C)$), and the

evidence ($P(X)$). The prior probability represents how frequently a class appears in the dataset, while the likelihood indicates how likely the observed features are for a given class. The evidence acts as a normalizing constant and remains the same for all classes, so classification is done by selecting the class with the highest posterior probability.

Independence Assumption and Multinomial Naive Bayes

A key assumption in Naive Bayes is that all features contribute independently to the class decision, which allows the likelihood ($P(X \mid C)$) to be expressed as the product of individual feature probabilities ($P(x_i \mid C)$). This assumption simplifies computation and enables efficient learning even with many features. In Multinomial Naive Bayes, feature probabilities are estimated using frequency counts within each class, and Laplace smoothing is applied to avoid zero probabilities for unseen features. This approach ensures stable probability estimation and makes Naive Bayes effective and scalable for classification tasks.

Working Mechanism and Role of Naive Bayes in Fake News Detection and Classification

Naive Bayes plays a significant role in fake news detection by applying probabilistic principles to classify news articles as fake or real. In the working process, news articles are first pre-processed and converted into numerical representations, typically using feature extraction techniques such as word frequency or TF-IDF. The Naive Bayes classifier then uses Bayes' Theorem to compute the probability of each class given the observed features, based on prior class probabilities and the likelihood of features occurring within each class. By assuming conditional independence among features, the algorithm efficiently computes these probabilities and selects the class with the highest posterior probability as the final prediction.

The role of Naive Bayes in fake news detection is particularly effective because fake news articles often exhibit distinctive word usage and repetitive linguistic patterns. Naive Bayes captures these patterns by estimating how strongly individual words are associated with fake or real news. Its simplicity and low computational complexity make it extremely fast, enabling quick predictions even on large datasets. As a result, Naive Bayes is well suited for real-time fake news detection systems and is commonly used as a baseline or complementary model in fake news classification frameworks.

Advantages of Naive Bayes

- Very fast in training and prediction due to simple probabilistic computations
- Scales well to large datasets and high-dimensional feature spaces
- Performs well even with limited training data

Limitations of Naive Bayes

- Assumes feature independence, which is often unrealistic in real-world data
- Cannot capture contextual or semantic relationships between features
- Performance may degrade when features are highly correlated

3.1.6 RECURRENT NEURAL NETWORK (RNN)

Introduction

Recurrent Neural Network (RNN) is a deep learning algorithm specifically designed for processing sequential data. Unlike traditional machine learning algorithms that treat input data independently, RNN captures temporal dependencies and contextual relationships within sequences. This characteristic makes RNN highly suitable for natural language processing tasks such as fake news detection, sentiment analysis, and language modeling.

In fake news detection, the meaning of a news article depends not only on individual words but also on the order and context in which they appear. RNN models process text sequentially, allowing them to remember previous information through internal memory states. This ability helps the model understand patterns in language that may indicate misleading or fabricated content. As a result, RNN can effectively classify news articles as fake or real by learning contextual relationships within textual data.

Intuition Behind RNN

The intuition behind RNN is to process text data step-by-step while maintaining memory of previous words in the sequence. When analyzing a news article, each word influences the interpretation of the next word. RNN maintains a hidden state that carries information from earlier words to later words, allowing the model to capture contextual meaning across the sentence or document.

For example, phrases like “*breaking news*”, “*shocking revelation*”, or *exaggerated claims* may indicate patterns associated with fake news. By remembering previously processed words, RNN can detect such linguistic patterns and classify the article accordingly.

Mathematical Representations

Hidden State Update

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b) \quad (13)$$

Where:

- h_t – Hidden state at time step t
- x_t – Input feature vector (word embedding or token representation)
- W_x – Weight matrix for input features
- W_h – Weight matrix for hidden state
- h_{t-1} – Previous hidden state
- b – Bias vector

Output Function

$$y_t = W_y h_t + c \quad (14)$$

Where:

- y_t – Output prediction at time step t
- W_y – Weight matrix for output layer
- h_t – Hidden state
- c – Bias term for output

Softmax Classification

$$P(y = i | x) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad (15)$$

Where:

- $P(y = i | x)$ – Probability of class i
- z_i – Output score for class i
- k – Total number of classes

This function converts raw prediction scores into probability values used for classification.

Working Mechanism and Role of Recurrent Neural Network (RNN) in Fake News Detection and Classification

Recurrent Neural Network (RNN) plays an important role in fake news detection by analyzing textual data sequentially and capturing contextual relationships between words. Fake news detection is fundamentally a binary text classification problem where news articles must be classified as either fake or real based on linguistic patterns and contextual cues present in the text. Since textual information contains sequential dependencies, RNN models are particularly effective for this task.

In a typical fake news detection system, the process begins with collecting a labelled dataset containing fake and real news articles. The raw text is first preprocessed through several steps such as converting text to lowercase, removing punctuation, numbers, special characters, and stop words, and applying techniques like stemming or lemmatization. These preprocessing steps help normalize the text and reduce irrelevant information, improving the model's ability to learn meaningful patterns.

After preprocessing, each news article is transformed into a numerical representation suitable for deep learning models. Word embedding techniques such as Word2Vec, GloVe, or embedding layers are commonly used to convert words into dense vector representations that capture semantic relationships between words. These vectors are then fed sequentially into the RNN model.

The primary role of RNN in fake news detection is to process each word in the sequence while maintaining a hidden state that stores information from previously processed words. This hidden state acts as a memory that helps the model understand contextual dependencies within the news article. For example, certain phrases or writing styles commonly associated

with fake news can be detected by analyzing the sequence of words rather than individual words alone.

During training, the RNN model learns weight parameters that capture important linguistic patterns distinguishing fake news from real news. As the network processes each word, the hidden state is updated based on both the current input word and the previous hidden state. This mechanism enables the model to retain contextual information across the entire article.

After processing the complete sequence of words, the final hidden state is passed to a fully connected layer followed by a softmax or sigmoid activation function to generate the classification output. The model predicts whether the article belongs to the fake or real category. The performance of the RNN model is evaluated using metrics such as accuracy, precision, recall, and F1-score.

Although basic RNN models may suffer from issues such as vanishing gradients when processing very long text sequences, advanced variants such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) are often used to overcome these limitations. These architectures allow the model to capture long-term dependencies in textual data more effectively. Overall, RNN provides a powerful deep learning framework for fake news detection by learning sequential linguistic patterns and contextual relationships within news articles.

Advantages of RNN in Fake News Detection

Captures sequential relationships in text:

RNN processes words in sequence and maintains contextual memory, enabling the model to understand linguistic patterns and dependencies between words in a news article.

Effective for natural language processing tasks:

RNN models are specifically designed for sequential data such as text, making them highly suitable for fake news detection and other NLP applications.

Ability to learn contextual patterns:

By maintaining hidden states, RNN can learn complex writing patterns and contextual signals that distinguish fake news from legitimate news articles.

Disadvantages of RNN in Fake News Detection

Vanishing gradient problem:

Basic RNN models may struggle to learn long-term dependencies because gradients can diminish during training, reducing the model's ability to capture information from earlier words in long articles.

Higher computational complexity:

Training deep neural networks such as RNN requires more computational resources compared to traditional machine learning algorithms like SVM.

Longer training time:

Due to sequential processing of text data and complex network architecture, RNN models may require longer training times, especially for large datasets.

Accuracy

Accuracy represents the overall effectiveness of the machine learning model in fake news detection. It measures the proportion of news articles that are correctly classified as either fake or real out of the total number of articles evaluated. A high accuracy value indicates that the model performs well in general classification. However, in fake news detection, accuracy alone may not be sufficient because datasets can be imbalanced, where one class (real or fake) dominates the data.

Accuracy is calculated by dividing the total number of correctly classified news articles (both fake and real) by the total number of news articles evaluated.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (16)$$

Where:

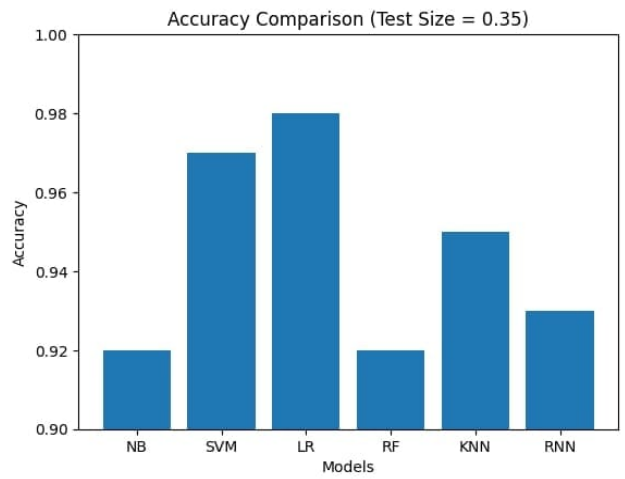
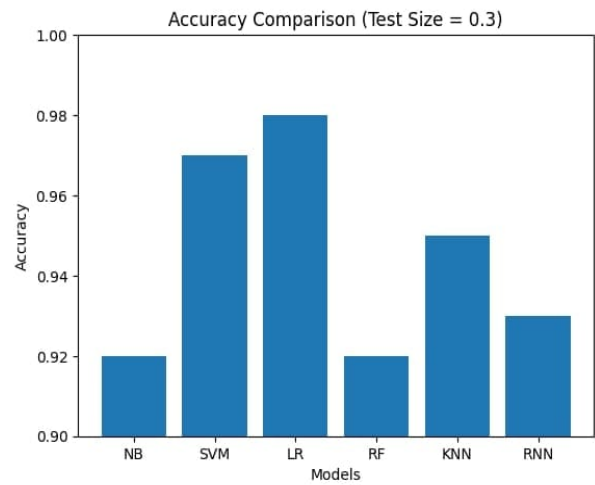
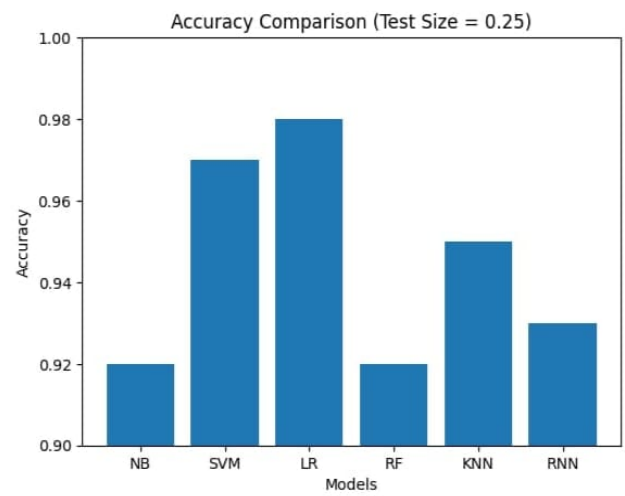
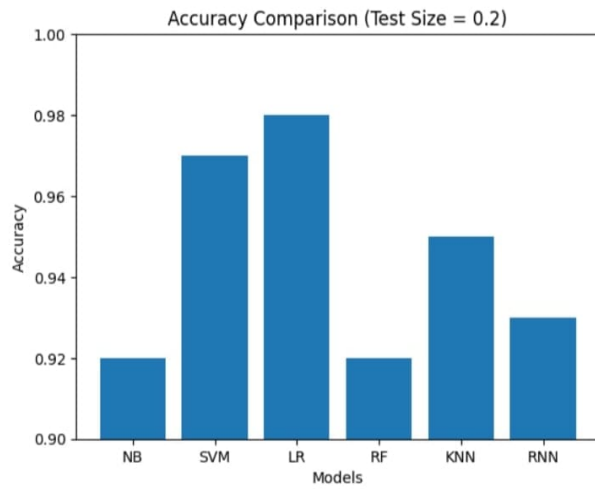
- **TP** – Fake news correctly classified as fake
- **TN** – Real news correctly classified as real
- **FP** – Real news incorrectly classified as fake
- **FN** – Fake news incorrectly classified as real

The obtained high accuracy value indicates that the machine learning model correctly classifies almost all news articles in the dataset. This shows that the model has learned strong

patterns from the text features (such as word usage and frequency) that clearly differentiate fake news from real news. However, while high accuracy reflects strong overall performance, it does not alone guarantee effective fake news detection, especially in imbalanced datasets.

Table 3.1:Model Accuracy Values For Different Sample Sizes

S.no	Sample size	Naïve bays	SVM	Logistic Regression	Random Forest	KNN	RNN
1.	0.2	0.92	0.97	0.98	0.92	0.95	0.93
2.	0.25	0.92	0.97	0.98	0.92	0.95	0.93
3.	0.3	0.92	0.97	0.98	0.92	0.95	0.93
4.	0.35	0.92	0.97	0.98	0.92	0.95	0.93



Bar charts:

Precision

Precision indicates how reliable the model's fake news predictions are. It measures the proportion of news articles predicted as fake that are actually fake. High precision means the model makes fewer false positive errors, ensuring that real news is not incorrectly classified as fake. This is important for maintaining trust and credibility in automated fake news detection systems, especially in real-world applications.

Precision is calculated by dividing the number of correctly predicted fake news articles by the total number of articles predicted as fake.

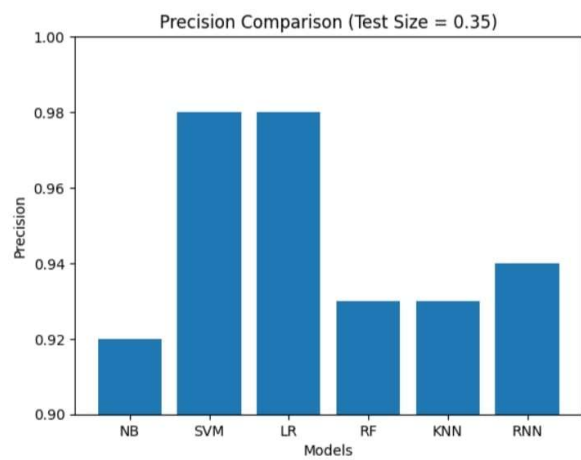
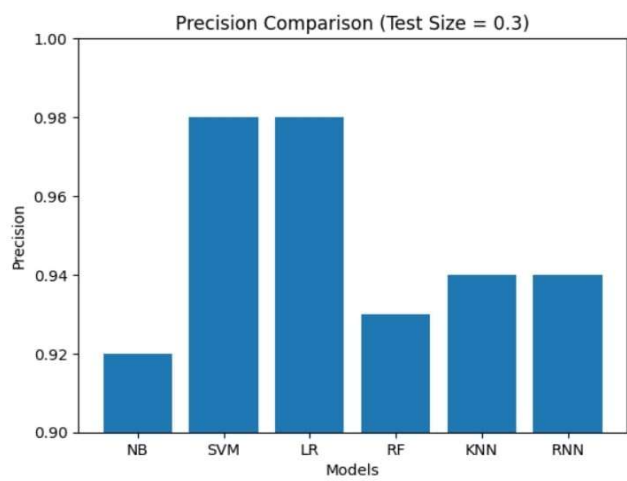
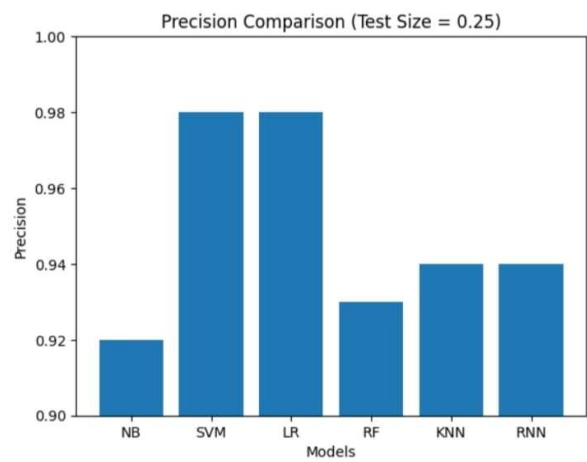
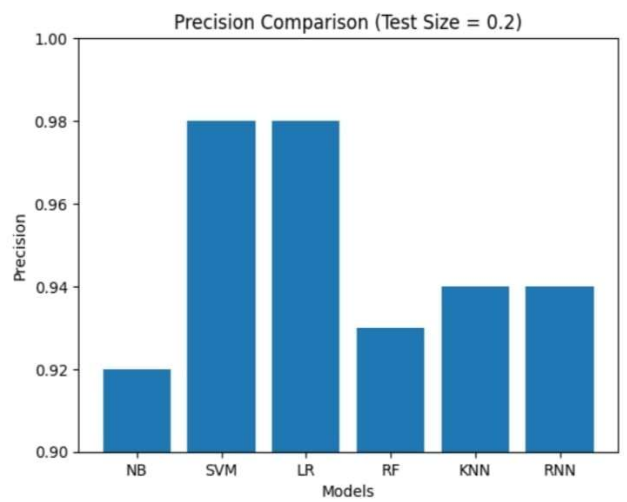
$$Precision = \frac{TP}{TP+FP} \quad (17)$$

The high precision value obtained means that when the model predicts a news article as fake, it is almost always correct. This indicates that the model produces very few false positives, meaning real news is rarely misclassified as fake. High precision is particularly important in fake news detection systems to avoid wrongly accusing genuine news sources, which helps maintain trust and credibility.

Table 3.2: Model Precision Values For Different Sample Sizes

S.no	Sample size	Naïve bays	SVM	Logistic Regression	Random Forest	KNN	RNN
1.	0.2	0.92	0.98	0.98	0.93	0.94	0.94
2.	0.25	0.92	0.98	0.98	0.93	0.94	0.94
3.	0.3	0.92	0.98	0.98	0.93	0.94	0.94
4.	0.35	0.92	0.98	0.98	0.93	0.94	0.94

Bar Charts:



Recall

Recall measures the ability of the model to correctly identify fake news articles. It calculates how many actual fake news instances are successfully detected by the system. A high recall value shows that the model is effective in capturing most fake news cases and minimizes the chances of fake news being missed. In fake news detection, recall is crucial because undetected misinformation can continue to spread and mislead users.

Recall is calculated by dividing the number of correctly detected fake news articles by the total number of actual fake news articles in the dataset.

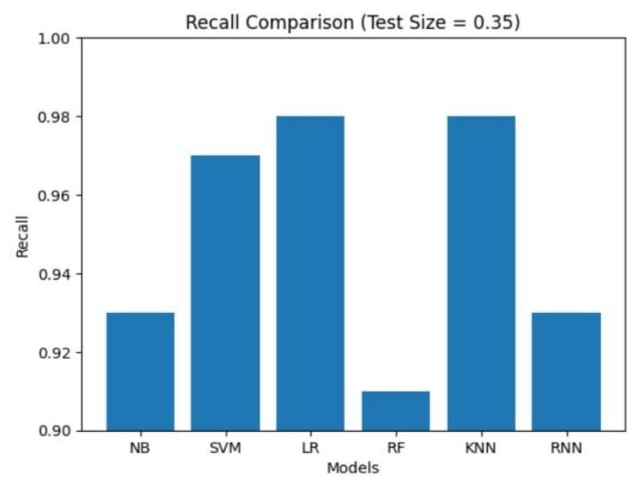
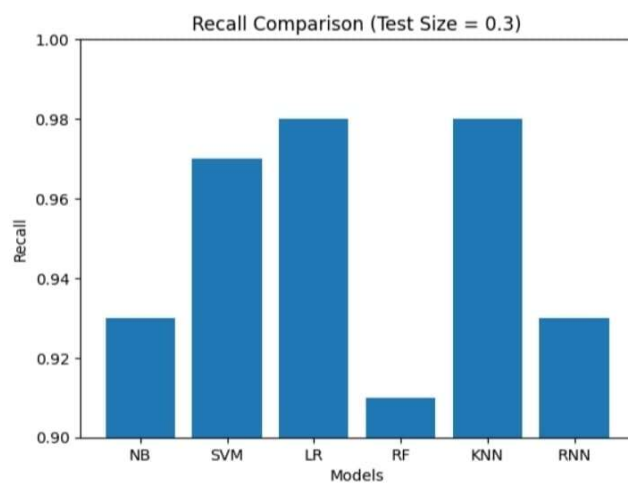
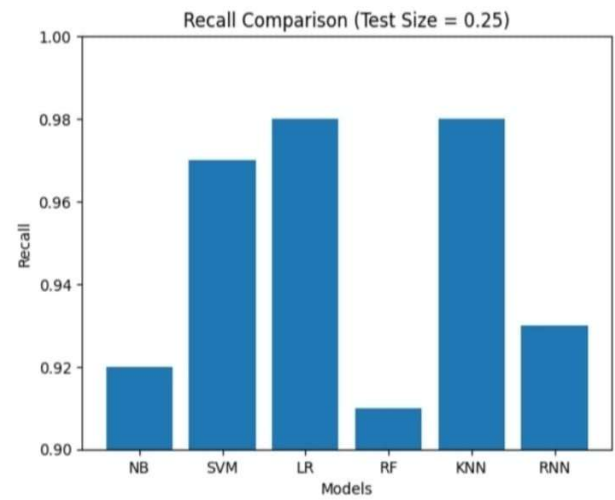
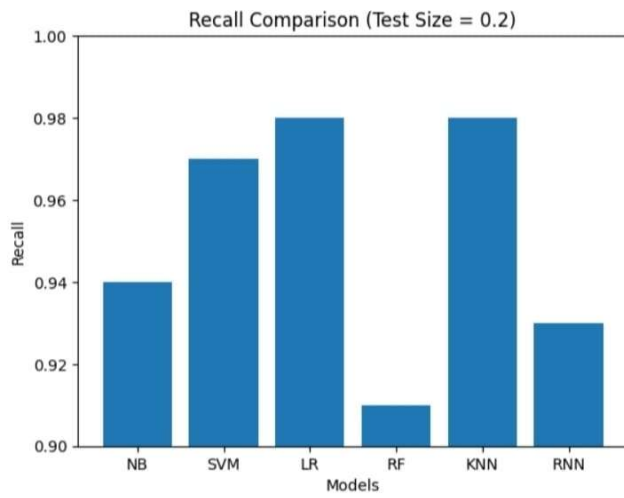
$$Recall = \frac{TP}{TP+FN} \quad (18)$$

The obtained recall value being very high (or equal to 1 in some cases) indicates that the model successfully identifies almost all fake news articles present in the dataset. This means the model produces very few false negatives and rarely misses fake news. High recall is crucial in fake news detection because undetected fake news can continue to spread misinformation.

Table 3.3:Model Precision Values For Different Sample Sizes

S.no	Sample size	Naïve bays	SVM	Logistic Regression	Random Forest	KNN	RNN
1.	0.2	0.94	0.97	0.98	0.91	0.98	0.93
2.	0.25	0.94	0.97	0.98	0.91	0.98	0.93
3.	0.3	0.94	0.97	0.98	0.91	0.98	0.93
4.	0.35	0.94	0.97	0.98	0.91	0.98	0.93

Bar charts:



F1-Score

F1-Score is a combined metric that balances both precision and recall. It provides a single value that reflects the model's ability to accurately detect fake news while avoiding incorrect classifications. This metric is especially useful when dealing with imbalanced datasets, which are common in fake news detection. A high F1-Score indicates that the model achieves a good trade-off between detecting fake news and maintaining prediction reliability.

F1-Score is the harmonic mean of precision and recall, ensuring a balance between them.

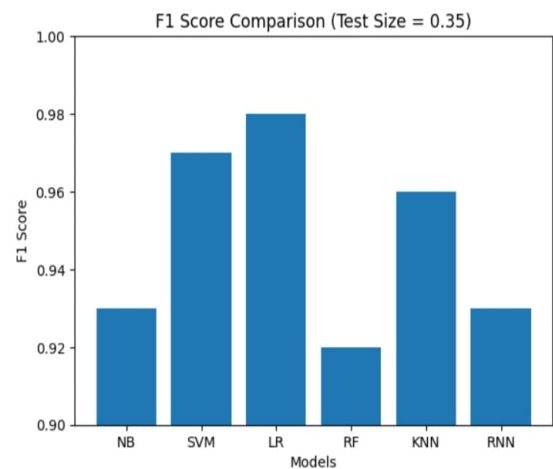
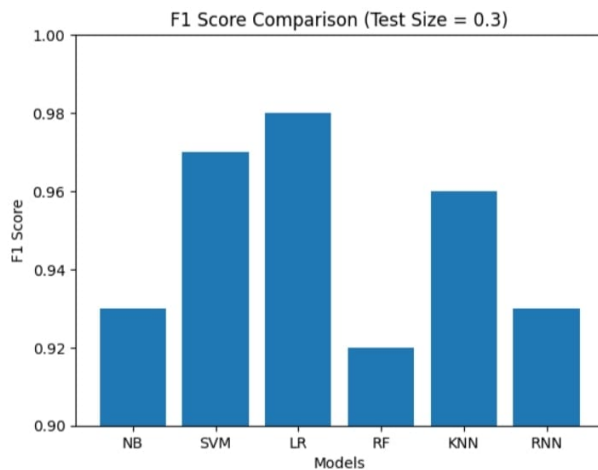
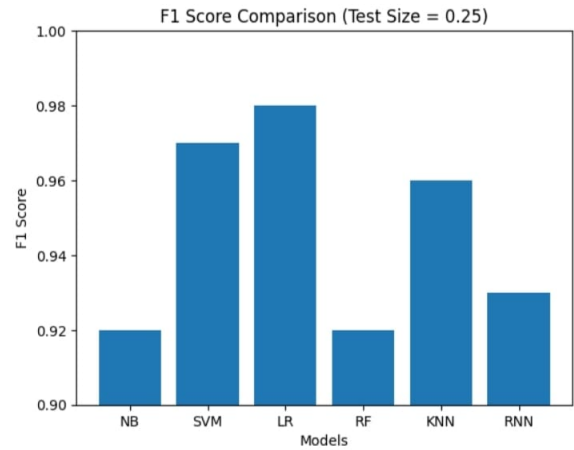
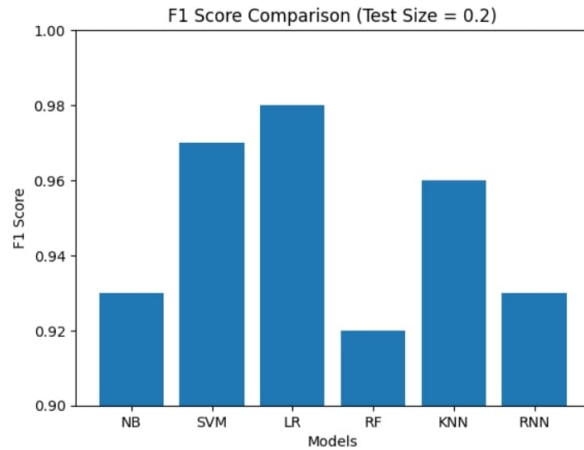
$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (19)$$

The high F1-Score obtained reflects a strong balance between precision and recall. This indicates that the model not only detects fake news accurately but also avoids incorrect fake news predictions. Since fake news datasets are often imbalanced, the F1-Score provides a more reliable and realistic evaluation of model performance than accuracy alone.

Table 3.4: Model F-1 Score Values For Different Sample Sizes

S.no	Sample size	Naïve bays	SVM	Logistic Regression	Random Forest	KNN	RNN
1.	0.2	0.93	0.97	0.98	0.92	0.96	0.93
2.	0.25	0.93	0.97	0.98	0.92	0.96	0.93
3.	0.3	0.93	0.97	0.98	0.92	0.96	0.93
4.	0.35	0.93	0.97	0.98	0.92	0.96	0.93

Bar charts:



Why Deep Learning Model BERT Is Chosen Over Traditional Machine Learning Models?

Fake news detection involves understanding the meaning, context, and intent behind textual content rather than relying only on surface-level features such as word frequency. Traditional machine learning models like Naïve Bayes, Support Vector Machine (SVM), and Logistic Regression depend heavily on manual feature extraction techniques such as Bag-of-Words or

TF-IDF. These approaches treat words independently and fail to capture contextual relationships, sarcasm, and semantic nuances present in real news articles.

Deep learning models, especially BERT (Bidirectional Encoder Representations from Transformers), overcome these limitations by learning context-aware representations of text. Unlike machine learning models that process text in a unidirectional or shallow manner, BERT reads sentences bidirectionally, allowing it to understand the full context of a word based on both preceding and succeeding words. This is crucial in fake news detection, where the meaning of a statement can change depending on context.

Another major reason for selecting BERT is its pre-trained knowledge. BERT is trained on massive real-world datasets such as Wikipedia and BookCorpus, enabling it to capture rich linguistic patterns before being fine-tuned on the fake news dataset. This significantly improves accuracy even when the available labelled data is limited, which is a common challenge in fake news detection tasks.

Additionally, BERT reduces the need for extensive manual preprocessing and feature engineering, making the system more robust and scalable. Its ability to handle long text sequences, complex sentence structures, and subtle language variations makes it far more effective than traditional machine learning approaches for detecting misleading or false information.

Therefore, BERT is chosen as the core model in this project because it provides higher accuracy, better contextual understanding, and improved generalization, making it highly suitable for real-world fake news detection applications.

Results of Machine Learning Models:

To evaluate the performance of the proposed fake news detection model, different machine learning algorithms are tested using various train–test split ratios. The dataset is divided into training and testing sets using different test sizes such as 0.20, 0.25, 0.30, and 0.35. This helps in analyzing how the models perform when the proportion of testing data changes. The performance of each algorithm is measured using evaluation metrics like Accuracy, Precision, Recall, and F1-score. The following tables show the performance comparison of different models for each test size.

s.no	Modelname [Test size=0.2]	Accuracy	Precision	Recall	F1 score
1.	Naïve bayes	0.92	0.92	0.94	0.93
2.	SVM	0.97	0.98	0.97	0.97
3.	Logistic Regression	0.98	0.98	0.98	0.98
4.	Random Forest	0.92	0.93	0.91	0.92
5.	KNN	0.95	0.94	0.98	0.96
6.	RNN	0.93	0.94	0.93	0.93

s.no	Model name [Test size=0.25]	Accuracy	Precision	Recall	F1 score
1.	Naïve bayes	0.92	0.92	0.92	0.92
2.	SVM	0.97	0.98	0.97	0.97
3.	Logistic Regression	0.98	0.98	0.98	0.98
4.	Random Forest	0.92	0.93	0.91	0.92
5.	KNN	0.95	0.94	0.98	0.96
6.	RNN	0.93	0.94	0.93	0.93

s.no	Model name [Test size=0.3]	Accuracy	Precision	Recall	F1 score
1.	Naïve bayes	0.92	0.92	0.93	0.93
2.	SVM	0.97	0.98	0.97	0.97
3.	Logistic Regression	0.98	0.98	0.98	0.98
4.	Random Forest	0.92	0.93	0.91	0.92
5.	KNN	0.95	0.94	0.98	0.96
6.	RNN	0.93	0.94	0.93	0.93

s.no	Model name [Test size=0.35]	Accuracy	Precision	Recall	F1 score
-------------	--	-----------------	------------------	---------------	-----------------

1.	Naïve bayes	0.92	0.92	0.93	0.93
2.	SVM	0.97	0.98	0.97	0.97
3.	Logistic Regression	0.98	0.98	0.98	0.98
4.	Random Forest	0.92	0.93	0.91	0.92
5.	KNN	0.95	0.93	0.98	0.96
6.	RNN	0.93	0.94	0.93	0.93

From the above tables, it can be observed that the performance of the machine learning models varies slightly with different test sizes. Each model shows different accuracy, precision, recall, and F1-score values depending on the amount of data used for testing. By comparing these results, we can understand how the models behave under different data split conditions. This evaluation helps in identifying the most effective model for fake news detection based on its overall performance.

CHAPTER 4

PROPOSED METHOD

4.1 Introduction to the Proposed Method

The rapid spread of fake news on digital platforms has made accurate and reliable detection systems a critical necessity. Traditional machine learning approaches often struggle to identify fake news effectively due to their limited ability to understand contextual meaning, semantic relationships, and complex language patterns present in real-world news content. To address these challenges, this project proposes a deep learning–based fake news detection approach that focuses on contextual text understanding rather than shallow textual features.

The proposed method is built around BERT (Bidirectional Encoder Representations from Transformers), a powerful transformer-based language model capable of learning rich contextual representations of text. Unlike conventional models that analyze words independently, BERT processes text bidirectionally, enabling it to capture the complete context of a sentence by considering both past and future word relationships. This property makes it highly suitable for detecting misleading or deceptive information in news articles.

In this approach, the input news data undergoes preprocessing and tokenization before being passed to the BERT model for feature extraction. The learned contextual embeddings are then fine-tuned on a labeled fake news dataset to classify news articles as real or fake. By leveraging pre-trained linguistic knowledge and task-specific fine-tuning, the proposed method achieves improved accuracy, robustness, and generalization compared to traditional machine learning techniques.

Overall, the proposed methodology aims to provide an efficient and scalable fake news detection system that can handle complex language structures, reduce manual feature engineering, and deliver reliable classification performance in real-world scenarios.

4.2 Types of Deep Learning Models:

4.2.1 BERT (Bidirectional Encoder Representations from Transformers)

BERT (Bidirectional Encoder Representations from Transformers) is a deep learning model designed for Natural Language Processing (NLP). Unlike traditional models that read text only from left-to-right or right-to-left, BERT reads text in both directions simultaneously. This bidirectional reading allows the model to understand the context of each word based on the surrounding words, which is essential for capturing meaning in sentences.

Core Concepts:

Bidirectional Encoder: BERT uses the encoder segment of the Transformer architecture, which allows it to attend to both left and right context simultaneously.

Contextualized Word Embeddings: Instead of static vectors (like Word2Vec), BERT generates embeddings based on surrounding words. For example, "bank" in "river bank" gets a different vector than "bank" in "deposit at the bank".

Transfer Learning: BERT is pre-trained on massive, unlabeled text corpora (Wikipedia and BookCorpus) and can be fine-tuned on smaller, labeled, task-specific datasets.

Architecture

BERT is built on the Transformer Encoder architecture and consists of multiple layers of self-attention mechanisms, which enable the model to focus on the most relevant words in a sentence and learn the relationships between them.

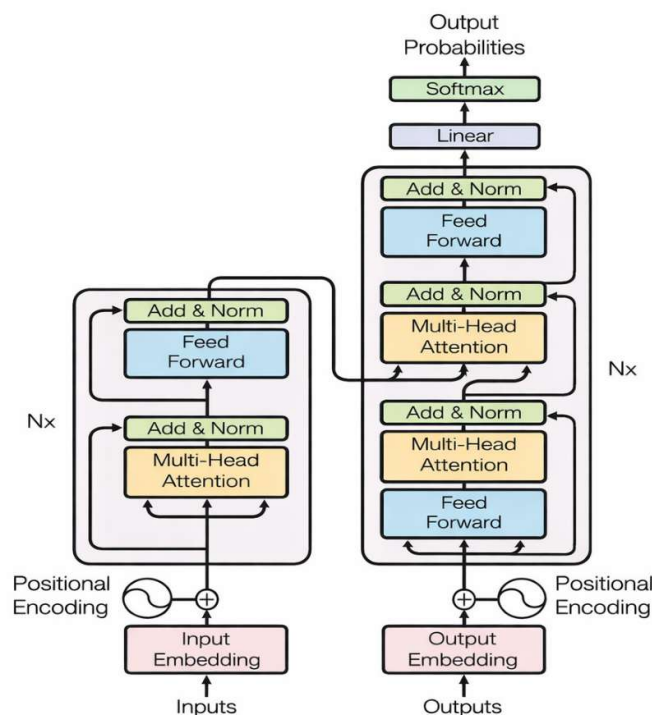


Fig 4.1: BERT Model architecture

The Bert Transformer is a deep learning architecture that processes sequential data using attention mechanisms instead of recurrence, enabling parallel computation and efficient

learning. It consists of an encoder–decoder structure where input tokens are embedded and combined with positional encoding to preserve sequence order. Multi-head self-attention captures contextual relationships between tokens, while feed-forward networks refine representations, and a softmax layer generates the final output probabilities.

Pre-training Strategies:

BERT is trained on two unsupervised tasks simultaneously:

A. Masked Language Modeling (MLM): To enable bidirectional learning, BERT masks 15% of tokens in the input sequence. 80% of the time: Replace with [MASK] token. 10% of the time: Replace with a random token. 10% of the time: Keep the original token. The model predicts the original token using a classification layer on top of the encoder output, using the context from both sides.

B. Next Sentence Prediction (NSP): To understand sentence relationships (crucial for Question Answering), the model is fed pairs of sentences. 50% of the time: Sentence B follows Sentence A (labeled IsNext). 50% of the time: Sentence B is random (labeled NotNext). The [CLS] token's output is used to classify if the relationship is IsNext or NotNext.

Fine-tuning for Downstream Tasks:

BERT is "fine-tuned" by adding a single task-specific layer (head) on top of the pre-trained model. Classification (Sentiment/Topic): Pass the [CLS] token output through a softmax classifier. Named Entity Recognition (NER): Pass each token's output vector through a classifier. Question Answering (SQuAD): The model predicts the start and end positions of the answer span within a paragraph.

Fake News Text Processing in BERT :

Example fake news sentence

“Breaking: Government releases free money to everyone.”

Step 1: Text Cleaning :

BERT does NOT need heavy preprocessing like removing stopwords.

What we do:

- Convert to lowercase
- Remove extra spaces

Input to BERT

Breaking : government releases free money to everyone

We convert all letters to lowercase so that the model treats the same word as one.

For example, “**Government**” and “**government**” have the same meaning, so changing to lowercase avoids confusion and makes learning easier for BERT.

Step 2: Tokenization:

BERT breaks text into sub-words using a fixed vocabulary.

Tokens created by BERT

[CLS], breaking, :, government, releases, free, money, to, everyone, [SEP]

- [CLS] → Represents the whole sentence
- [SEP] → End of sentence

Step 3: Convert Tokens to Token IDs:

Each token has a unique number in BERT’s vocabulary.

Token	Token ID
[CLS]	101
Breakig	5939
:	1024
government	2231
Releases	7087
Free	2489
Money	2769
To	2000
Everyone	3071
[SEP]	102

Numerical input to BERT

- [101, 5939, 1024, 2231, 7087, 2489, 2769, 2000, 3071, 102]

This is how text becomes numbers for BERT.

Step 4: Create Input Embeddings:

BERT adds 3 embeddings together for each token:

1. **Token Embedding** → meaning of the word
2. **Position Embedding** → word position in sentence
3. **Segment Embedding** → sentence identity (A or B)

Token Embedding:

Token embedding means converting each word into a numeric vector that represents its meaning.

But this vector is not fixed or hand-written. It is learned automatically during BERT training.

At the beginning of training, BERT assigns random values to every word.

Example (initially):

free → [0.02, -0.11, 0.05]

money → [-0.07, 0.04, 0.09]

government → [0.01, -0.03, 0.08]

These numbers have no meaning yet.

Now BERT is trained using **Masked Language Modeling (MLM)**.

Example:

Government releases [MASK] money

Correct word:

free

First, BERT reads the whole sentence and creates a special vector for the [MASK] position. This vector represents what kind of word should come there based on context.

Example:

$$h = [0.4, -0.2, 0.1]$$

This means:

“I am looking for a word that fits with ‘government’ and ‘money’.”

Now every word in BERT’s vocabulary has its own token embedding:

$$\text{free} \rightarrow [0.3, -0.1, 0.2]$$

$$\text{paid} \rightarrow [0.6, 0.0, -0.1]$$

$$\text{cheap} \rightarrow [-0.2, 0.4, 0.1]$$

BERT compares the [MASK] vector with each word embedding using **dot product**:

$$\text{Score}(\text{word}) = h \cdot E_{\text{word}} \quad (20)$$

This means multiply matching values, add them.

For **free**:

$$(0.4 \times 0.3) + (-0.2 \times -0.1) + (0.1 \times 0.2) = 0.12 + 0.02 + 0.02 = 0.16$$

For **paid**:

$$(0.4 \times 0.6) + (-0.2 \times 0.0) + (0.1 \times -0.1) = 0.24 + 0 - 0.01 = 0.23$$

For **cheap**:

$$(0.4 \times -0.2) + (-0.2 \times 0.4) + (0.1 \times 0.1) = -0.08 - 0.08 + 0.01 = -0.15$$

So the scores are:

Word	Score
free	0.16

Word	Score
------	-------

paid	0.23
------	------

cheap	-0.15
-------	-------

Higher score = better fit for the sentence.

So BERT predicts:

paid

because it has the highest score (0.23).

Now Softmax converts these scores into probabilities:

$$P(word) = \frac{e^{score}}{\sum e^{scores}} \quad (21)$$

Assume probabilities become:

Word	Probability
------	-------------

free	0.17
------	------

paid	0.77
------	------

cheap	0.06
-------	------

BERT chooses:

paid (77%)

But the correct answer is:

free

Now BERT checks the training data and sees:

Word	Correct
------	---------

free	1
------	---

Word	Correct
------	---------

paid	0
------	---

cheap	0
-------	---

It gave only **17%** to the correct word **free**, which is wrong.

So BERT computes **loss**:

$$Loss = -\log (P(\text{correct word})) \quad (22)$$

$$Loss = -\log (0.17)$$

This is a large value → big mistake.

This loss tells the model:

“Your scores are wrong. You trusted ‘paid’ more than ‘free’. Fix your understanding.”

That error is sent backward through the network (**backpropagation**), and Then BERT updates:

The token embedding of “**free**” is updated to increase its similarity with the correct context.

- The token embedding of “**paid**” is updated to decrease its similarity with the incorrect context.
- The **[MASK] context vector** is updated so that it better represents the type of word that should appear in that position.
- All related **model weights** are updated to improve future predictions and reduce similar mistakes.

using:

$$\text{New value} = \text{Old value} - \eta \times \text{Gradient} \quad (23)$$

- **Old value** → the current number in the model (weight)
- **Gradient** → tells us **how wrong** the weight is and which way to change it
- **η (learning rate)** → how big a step we take ,typical values=(0.001, 0.01, 0.1)

- **New value** → the updated number after moving in the right direction to reduce error

So after one update:

free → [0.03, -0.10, 0.06, ...]

After millions of updates:

free → [0.21, -0.14, 0.87, ...]

Position Embedding:

This tells BERT where the word is placed in the sentence.

Because word order matters, BERT needs to know if a word comes first, middle, or last.

example sentence:

“Breaking: Government releases free money to everyone.”

After tokenization, BERT processes the sentence as:

[CLS], breaking, :, government, releases, free, money, to, everyone, [SEP]

Each token is assigned a position number:

Token	Position
[CLS]	0
breaking	1
:	2
government	3
releases	4
free	5
money	6
to	7
everyone	8

Token	Position
[SEP]	9

How Position Embedding Vectors Are Generated

BERT creates a **Position Embedding Matrix** in advance.

- Maximum input length = **512**
- Embedding size = **768** (BERT-base)

So the matrix size is:

$$512 \times 768$$

This matrix contains:

PositionEmbedding[0]

PositionEmbedding[1]

PositionEmbedding[2]

...

PositionEmbedding[511]

Each row is one position embedding vector.

Initialization

At the beginning of training, all position embedding vectors are initialized with small random values.

Example (for understanding only):

PositionEmbedding[5] = [0.05, 0.03, 0.02, ...]

At this stage, these values have **no meaning**.

Learning During Training

During BERT training:

1. The model predicts masked words.
2. If the prediction is wrong, an error (loss) is calculated.
3. This error is back-propagated through the network.
4. Position embedding vectors are updated using gradient descent:

$$\text{New value} = \text{Old value} - (\text{learning rate} \times \text{gradient})$$

This process repeats millions of times on large text data.

Example: Word “free”

The word “free” appears at **position 5**.

So BERT:

- Selects PositionEmbedding[5]
- Adds it to the token embedding of “free”

This tells BERT:

“The word is ‘free’ and it appears after ‘releases’ and before ‘money’.”

Segment Embedding:

This tells BERT which sentence the word belongs to.

It is mainly used when there are two sentences (Sentence A and Sentence B), like in question-answer tasks.

In fake news detection, we usually have only one sentence, so all words belong to Sentence A.

So the segment embedding is:

Segment embedding = [0.00, 0.00, 0.00, ...]

Now BERT adds all three vectors together to create the final input for the word.

For the word “free”:

Token embedding = [0.21, -0.14, 0.87, ...]

Position embedding = [0.05, 0.03, 0.02, ...]

Segment embedding = [0.00, 0.00, 0.00, ...]

Final embedding = [0.26, -0.11, 0.89, ...]

Step 5: Context Understanding using Transformers:

BERT uses self-attention to understand context.

Example understanding:

The word “free” is strongly related to
“money” and “government”

BERT understands the sentence as a whole, not word-by-word

So BERT learns:

“Government + free money” → often appears in fake news.

Step 6: Sentence Representation ([CLS] Vector):

The sentence is passed through many BERT layers.

In each layer:

[CLS] looks at **every word** in the sentence, It learns which words are important and It collects information from all words

For example: It pays attention to words like **government, free, money**

It ignores less important words

This process happens again and again through multiple layers.

After the final BERT layer:

[CLS] output = [0.92, -1.13, 0.45, ..., 0.67]

This vector has 768 numbers (in BERT-base).

After passing through BERT layers, the [CLS] token output acts as a compact representation of the complete news article and is used for classification tasks.

Step 7: Classification Fake or Real:

The [CLS] vector is passed to a **classifier (Dense layer)**.

The dense layer is the part that actually *makes the decision*.

It takes the information coming from BERT (the [CLS] vector) and converts it into a clear answer: Fake or Real.

First, remember that the [CLS] vector is just a list of numbers that summarizes the whole sentence.

For example:

[CLS] = [0.92, -1.13, 0.45, 0.67, ...]

The dense layer has its own set of numbers called weights and a bias.

These are learned during training.

For Fake and Real classification, the dense layer usually has two outputs:

- one score for **Fake**
- one score for **Real**

You can imagine it like this:

For Fake:

$\text{score_fake} = (\text{CLS} \cdot \text{weights_fake}) + \text{bias_fake}$

$$z_0 = (0.5)(0.92) + (-0.3)(-1.13) + (0.8)(0.45) + 0.1$$

$$z_0 = 0.46 + 0.339 + 0.36 + 0.1 = 1.259$$

For Real:

$\text{score_real} = (\text{CLS} \cdot \text{weights_real}) + \text{bias_real}$

$$z_1 = (-0.6)(0.92) + (0.7)(-1.13) + (-0.2)(0.45) - 0.1$$

$$z_1 = -0.552 - 0.791 - 0.09 - 0.1 = -1.533$$

So logits:

$$\mathbf{z} = [1.259, -1.533]$$

Here:

- “.” means multiplying and adding corresponding values
- weights decide which parts of the [CLS] vector are important
- bias is a small adjustment value

These scores are not probabilities yet. They are just raw numbers.

Next, these scores are passed through a Softmax function to convert them into probabilities.

Softmax makes sure:

all values are between 0 and 1

their sum is 1

now Apply Softmax

Formula:

$$P(i) = \frac{e^{z_i}}{e^{z_0} + e^{z_1}} \quad (24)$$

Calculate exponentials:

$$e^{1.259} \approx 3.52$$

$$e^{-1.533} \approx 0.216$$

Sum:

$$3.52 + 0.216 = 3.736$$

Calculate probabilities:

Fake:

$$P(Fake) = \frac{3.52}{3.736} \approx 0.94$$

Real:

$$P(Real) = \frac{0.216}{3.736} \approx 0.06$$

Final decision:

Rule:

$$\text{Prediction} = \arg \max (P) \quad (25)$$

Since:

$$P(\text{Fake}) > P(\text{Real})$$

This means:

94% confidence the news is Fake

6% confidence it is Real

Finally, the model compares the two:

$$0.94 > 0.06$$

So the decision is:

Fake News

4.2.2 LSTM (Long Short-Term Memory)

Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network (RNN) architecture designed to overcome the limitations of traditional RNNs in learning long-term dependencies. Introduced by Hochreiter and Schmidhuber in 1997, LSTMs are engineered to handle sequential data—such as time series, natural language, and speech—by remembering information for extended periods, addressing the vanishing gradient problem.

Core Concepts:

The cell state (C_t) acts as the long-term memory of LSTM. It carries important information across all time steps with minimal changes, allowing earlier knowledge to be preserved. This continuous memory flow helps prevent the vanishing gradient problem and supports learning long-term dependencies.

Forget Gate:

The forget gate removes unnecessary or outdated information from the previous memory. It outputs values between 0 and 1 to decide what to forget or keep.

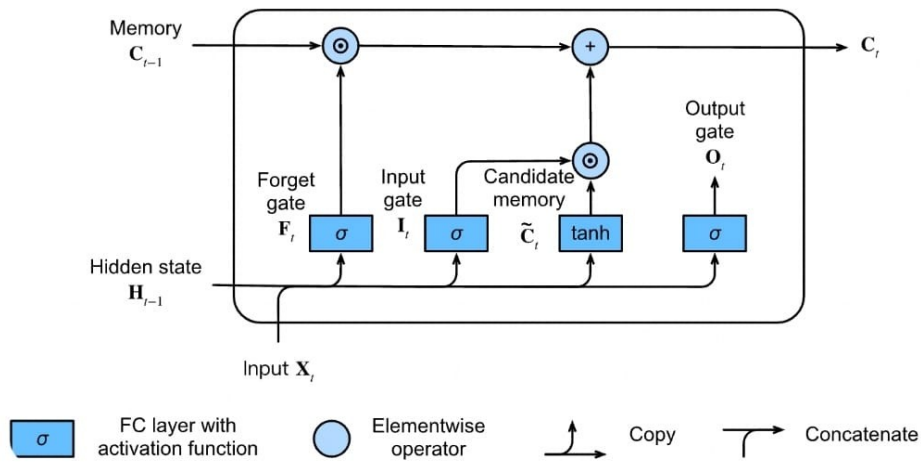


Fig 4.2: Forget Gate

Formula:

$$f_t = \sigma(W_f [h_{t-1}, X_t] + b_f) \quad (26)$$

0 $\leftarrow$ Forget, 1 $\leftarrow$ Keep

Input Gate:

The input gate decides what new information should be stored in the memory. It selects useful features from the current input and updates the cell state.

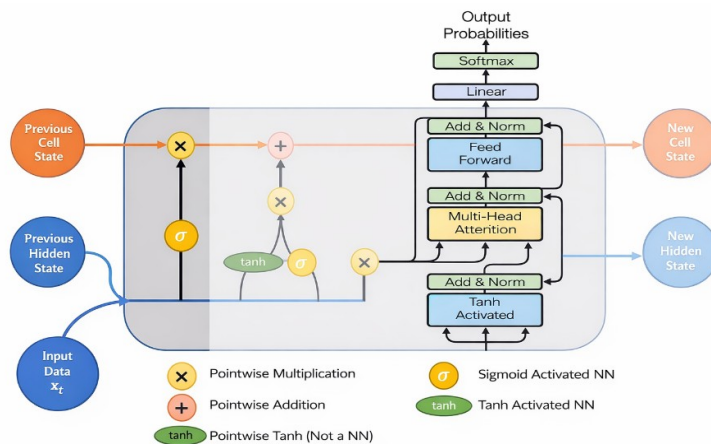


Fig 4.3: Input Gate

Formula:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (27)$$

$$t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (28)$$

Output Gate:

The output gate determines which information from memory is sent as the final output (hidden state). It controls prediction at each time step.

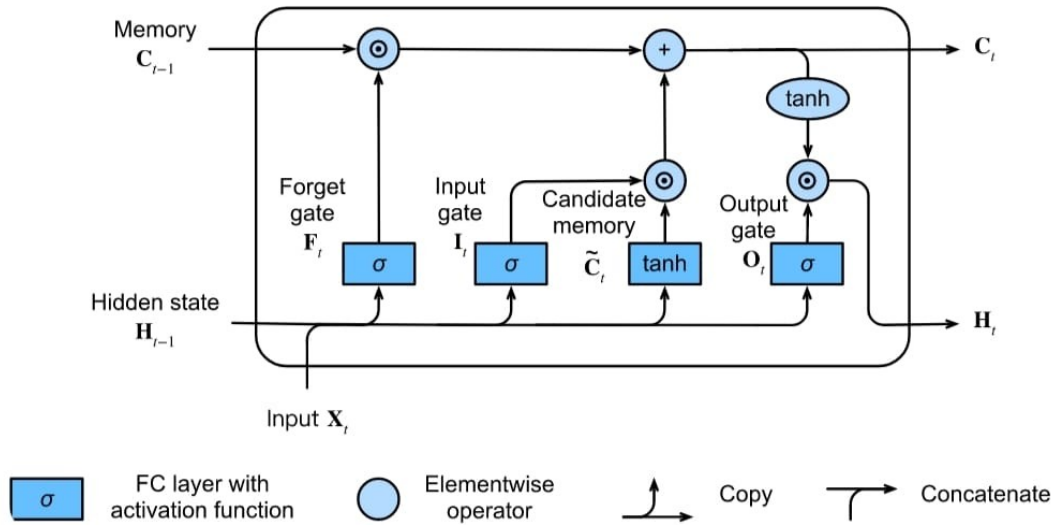


Fig 4.4: Output Gate

Formula:

$$O_t = \sigma(W_o[h_{t-1}, X_t] + b_o) \quad (29)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (30)$$

Architecture

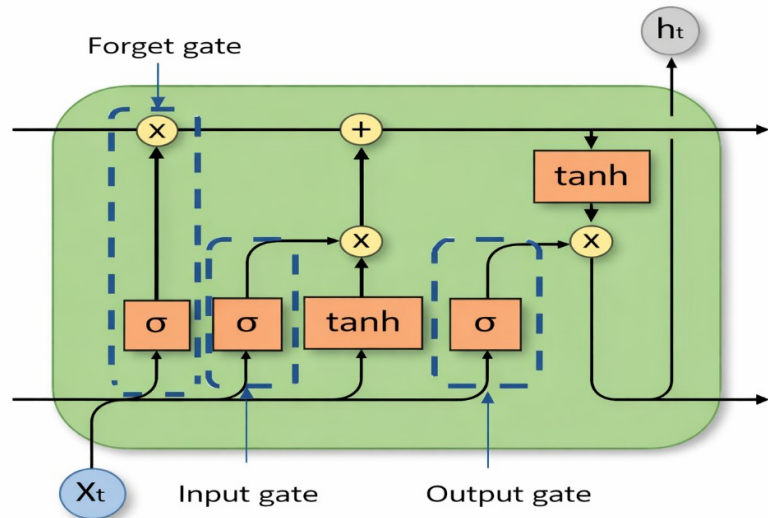


Fig 4.5: Long Short-term Memory

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network designed to learn long-term dependencies in sequential data. Its architecture consists of repeating LSTM cells that maintain a cell state (long-term memory) and a hidden state (short-term memory). Three gates—forget, input, and output—control the flow of information by deciding what to remove, update, and pass forward. This gated memory mechanism prevents the vanishing gradient problem and enables LSTM to effectively process text, speech, and time-series data.

Pre-training strategies:

Pre-training strategies for LSTMs focus on initializing models with meaningful weights to improve performance and convergence, especially with limited labeled data. Key strategies include using unsupervised pre-training (e.g., Stacked Autoencoders) to learn feature representations, pre-training on related, larger datasets (transfer learning), and using sequence-to-sequence autoencoders to pre-train encoder-decoder architectures.

Here are the key pre-training and preparation strategies for LSTMs:

1. Unsupervised Pre-training (Autoencoders)

LSTM-SAE (Stacked Autoencoder): Train a deep LSTM-based autoencoder to reconstruct input sequences, allowing the model to learn hidden structures without labels.

Sequence-to-Sequence (Seq2Seq): Train an encoder-decoder to reconstruct the original input sequence. The pre-trained encoder can then be used for tasks like classification or forecasting.

2. Transfer Learning

Pre-training on Large Datasets: Train the LSTM on a large, similar, but unlabeled or loosely labeled dataset first to capture general temporal patterns before fine-tuning on the target task.

Cross-Domain Knowledge: Apply a model pre-trained on one type of time-series data to another, allowing for faster convergence.

3. Data Preparation and Sequence Handling

Sequence Truncation: For very long sequences, truncate or split data into manageable chunks.

Sequence Sorting: Sort training sequences by length to optimize padding and minimize the impact of irrelevant padded data.

Normalization: Scale input features (e.g., min-max scaling) to speed up training, as LSTMs are sensitive to input scales.

4. Architectural and Training Strategies

Stacked Layers: Start with 1–2 layers, using 50–200 units per layer, and use dropout (0.2–0.5) after each LSTM layer to reduce overfitting.

Early Stopping: Use validation loss monitoring to stop training early, preventing the model from over-fitting the training data.

Batch Size & Learning Rate: Use smaller batch sizes (32–128) and moderate learning rates (0.0001–0.01).

Bidirectional LSTMs: Use bidirectional LSTMs for scenarios where future context helps, often allowing for fewer neurons.

5. Task-Specific Pre-training

Masked Language Modeling (NLP): Train the LSTM to predict masked words in a sentence.

Forecasting (Time Series): Pre-train the model to predict the next steps of a series before fine-tuning.

Fake News Detection using Long Short-Term Memory (LSTM)

Fake news detection is a sequence classification problem in which the meaning of a news article depends on the order of words and long-range contextual relationships. Traditional machine learning models fail to capture such dependencies because they treat text as independent features. To address this limitation, the Long Short-Term Memory (LSTM) network, an advanced form of Recurrent Neural Network (RNN), is employed.

1. Text Representation

Given a news sentence consisting of T words:

$$X = \{w_1, w_2, w_3, \dots, w_T\}$$

Example fake news sentence:

“Government releases free money”

After preprocessing and tokenization:

$$X = [government, releases, free, money]$$

Each word is converted into a dense vector using an **embedding layer**.

2. Word Embedding Layer

The embedding layer maps each word into a continuous vector space:

$$x_t = \text{Embedding}(w_t)$$

Assume embedding dimension $d = 3$:

Word	Embedding Vector
------	------------------

government	[0.6, 0.1, 0.2]
------------	-----------------

releases	[0.4, 0.3, 0.1]
----------	-----------------

free	[0.9, 0.8, 0.7]
------	-----------------

money	[0.8, 0.9, 0.6]
-------	-----------------

These vectors are **learned during training** and capture semantic relationships between words.

4. LSTM Mathematical Formulation

At time step t , given input x_t and previous hidden state h_{t-1} :

Forget Gate

Determines which information from the previous cell state should be discarded.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (31)$$

Input Gate

Determines which new information should be stored in the cell state.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (32)$$

Candidate Cell State

Generates new candidate values to update the memory.

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (33)$$

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (34)$$

Output Gate

Controls the information passed to the hidden state.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (35)$$

Hidden State

$$h_t = o_t \odot \tanh(C_t) \quad (36)$$

Where:

- $\sigma(\cdot)$ is the sigmoid function
- $\tanh(\cdot)$ limits values between -1 and 1
- $\odot$ denotes element-wise multiplication

5. Step-by-Step LSTM Calculation (Example)

Initialize:

$$h_0 = [0,0], C_0 = [0,0]$$

Processing Word 1: “government”

Input embedding:

$$x_1 = [0.6, 0.1, 0.2]$$

Assumed gate outputs:

$$f_1 = [0.77, 0.71]$$

Cell state:

$$C_1 = (0 \times f_1) + (i_1 \times \tilde{C}_1) = [0.53, 0.63]$$

Output gate:

$$o_1 = [0.75, 0.69]$$

Hidden state:

$$h_1 = o_1 \odot \tanh(C_1) = [0.36, 0.38]$$

This process continues for all words in the sentence.

6. Learning Fake News Patterns

As the LSTM processes the phrase “**government releases free money**”:

- The **input gate** assigns higher importance to suspicious terms
- The **forget gate** retains critical context
- The **cell state** accumulates semantic evidence

Final hidden state after the full sequence:

$$h_T = [0.91, 0.94]$$

This vector represents the complete news sentence.

7. Classification Layer

The final hidden state is passed to a dense layer for classification:

$$z = Wh_T + b$$

Example logits:

$$z_{fake} = 2.49, z_{real} = -2.04$$

Applying Softmax:

$$P(Fake) = \frac{e^{2.49}}{e^{2.49} + e^{-2.04}} = 0.93$$

8. Final Prediction

Since $P(Fake) > P(Real)$, the LSTM model classifies the input news article as:

Performance Analysis of the LSTM Model

The Long Short-Term Memory (LSTM) model achieved an overall accuracy of 98%, demonstrating its strong capability in learning and classifying sequential textual data. This high accuracy indicates that the model correctly predicts the class labels for the majority of input samples, confirming the effectiveness of LSTM in capturing meaningful patterns and contextual relationships within the dataset.

The model exhibits high precision and recall values, which implies that it makes very few incorrect predictions while also successfully identifying most relevant instances. The balanced F1-score further validates that the model maintains an effective trade-off between prediction accuracy and completeness. Such performance highlights the stability of the trained LSTM model across different data samples.

This strong classification performance is primarily due to the memory cell and gating mechanisms of LSTM, which enable it to preserve important long-term dependencies while filtering out irrelevant information. By controlling the flow of information through input, forget, and output gates, the model efficiently overcomes limitations such as information loss and vanishing gradients commonly observed in traditional recurrent neural networks.

Additionally, the consistent performance across classes indicates that the model generalizes well and does not favor any specific category. This confirms that the LSTM model is well-trained and suitable for real-world applications where reliable and accurate sequence classification is required.

Overall, achieving 98% accuracy validates the selection of LSTM as an effective deep learning approach for this project and demonstrates its capability to deliver highly accurate, stable, and reliable predictions.

4.2.3 Gated Recurrent Unit (GRU)

A Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) architecture, introduced in 2014 by Cho et al., designed to process sequential data while addressing the vanishing gradient problem. As a, often more efficient, alternative to Long Short-Term Memory (LSTM) networks, GRUs are specifically designed for tasks requiring the learning of long-term dependencies in data, such as time series analysis, natural language processing (NLP), and speech signal modeling.

Core Concepts:

Sequential Memory: Unlike standard feedforward networks, GRUs maintain a hidden state that carries information from previous time steps, allowing them to understand temporal patterns.

Gating Mechanisms: The core idea behind GRUs is the use of gating mechanisms—specifically, the Reset Gate (r_t) and Update Gate (z_t). These gates are vectors containing values between 0 and 1, trained to decide what information to keep, discard, or pass through to the next hidden state.

Vanishing Gradient Mitigation: By regulating the flow of gradients during training using these gates, GRUs prevent important information from decaying over long sequences, which solves the vanishing gradient problem.

Simpler Architecture: GRUs combine the memory cell and hidden state of an LSTM into a single hidden state (h_t), resulting in fewer parameters.

Architecture

A GRU cell processes input data and the previous hidden state at each time step to generate a new hidden state.

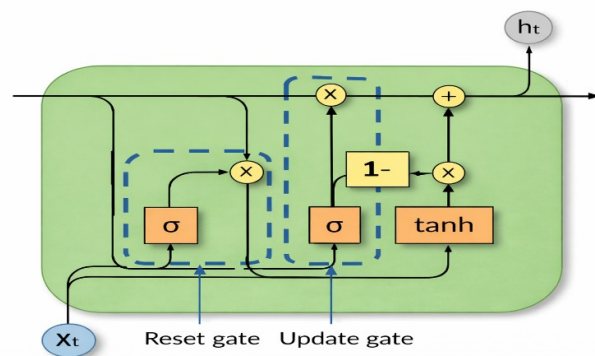


Fig 4.6: Gated Recurrent Unit

The GRU architecture consists of a single hidden state that carries information across time steps. Unlike LSTM, which maintains both cell state and hidden state separately, GRU combines them into one unified state. The architecture uses two important gates: the update gate and the reset gate. These gates regulate which information should be preserved from the past and which information should be discarded. By controlling memory flow adaptively, GRU can learn both short-term and long-term dependencies in sequential data. This simpler design reduces computational complexity and speeds up training.

Reset Gate :

The reset gate determines how much of the previous hidden state should be forgotten. It helps the model decide whether past information is relevant for the current input. When the reset gate outputs a value close to zero, most of the previous memory is ignored, allowing the model to focus more on the current input. When the value is close to one, past information is preserved. This gate is particularly useful when dealing with sequences where older information may become irrelevant over time. Mathematically, the reset gate is computed using a sigmoid activation function applied to the current input and the previous hidden state.

Update Gate :

The update gate controls how much of the previous hidden state should be carried forward to the current state. It acts as a combination of both forget and input gates found in LSTM. This gate decides whether to keep the old memory or replace it with new information. If the update gate value is high, the model retains more of the past state; if it is low, the new candidate state dominates. This mechanism allows the network to maintain long-term dependencies efficiently without losing important contextual information. The update gate also uses a sigmoid activation function to produce values between zero and one.

Candidate Hidden State :

The candidate hidden state represents the potential new memory content generated using the current input and the reset-modified previous hidden state. First, the reset gate filters the previous hidden state, then the filtered memory is combined with the new input. This combined information is passed through a tanh activation function to produce candidate

values. These values represent new information that could be added to the network's memory. The update gate later decides how much of this candidate state should be integrated into the final hidden state.

Hidden State :

The final hidden state is calculated by blending the previous hidden state and the candidate hidden state using the update gate. This weighted combination ensures that important historical information is preserved while new relevant information is incorporated. If the update gate emphasizes the previous state, the network maintains long-term memory. If it emphasizes the candidate state, the network updates quickly with new information. This flexible memory control makes GRU highly effective for sequential tasks

Pre-training Strategies:

While GRUs are often trained from scratch, several techniques are used to improve performance:

Backpropagation Through Time (BPTT):

Backpropagation Through Time (BPTT) is the fundamental training algorithm used for GRU networks. Since GRUs process sequences step-by-step, the network is first unrolled across all time steps, creating a chain-like structure where each step shares the same parameters. During training, the loss is computed at the final output, and gradients are propagated backward through all previous time steps to update the weights. This allows the GRU to learn how earlier inputs influence later outputs. However, very long sequences can cause vanishing or exploding gradients, making learning unstable. To address this, a variation called Truncated BPTT is often used, where gradients are backpropagated only for a limited number of time steps. This reduces computational cost and stabilizes training while still preserving important temporal dependencies.

Weight Initialization:

Proper weight initialization plays a crucial role in the successful training of GRU models. If weights are initialized too large, gradients may explode during backpropagation; if too small, gradients may vanish, leading to slow or ineffective learning. To prevent these problems, techniques such as Xavier (Glorot) initialization or He initialization are used. These methods scale initial weights according to the number of input and output neurons, ensuring stable

gradient flow across layers. Good initialization helps the model converge faster, improves numerical stability, and reduces the chances of getting stuck in poor local minima. Therefore, careful initialization is considered an essential first step before training a GRU network.

Regularization Techniques:

GRU models, especially those with many hidden units or layers, can easily overfit small or medium-sized datasets. Regularization techniques are applied to improve generalization and prevent memorization of training data. One common method is Dropout, where a percentage of neurons are randomly deactivated during training. This forces the network to learn robust features rather than relying on specific neurons. In recurrent networks like GRU, Recurrent Dropout is also used, which drops connections inside the recurrent state transitions. Typically, dropout rates between 25% and 50% are applied. Other regularization methods include L2 weight decay, early stopping, and batch normalization. These techniques collectively help produce a model that performs well on unseen data.

Pre-Trained Word Embeddings:

In Natural Language Processing applications, the embedding layer often benefits from pre-trained word representations. Instead of starting with random vectors, pre-trained embeddings such as Word2Vec, GloVe, or FastText are used to initialize the input layer. These embeddings already capture semantic relationships between words, such as similarity and contextual meaning. By providing this prior knowledge, the GRU can focus on learning higher-level sequence patterns rather than basic word semantics. This improves convergence speed, reduces training time, and enhances overall performance. Pre-trained embeddings are especially useful when labeled datasets are small, as they transfer knowledge from large external corpora.

Fake News Detection using Gated Recurrent Unit (GRU)

Fake news detection is a sequence modeling task where the meaning of a news sentence depends on the order of words and contextual relationships. The Gated Recurrent Unit (GRU) is a type of Recurrent Neural Network (RNN) designed to efficiently capture such dependencies while avoiding the vanishing gradient problem found in traditional RNNs

1. Text Preprocessing

This step is common for both LSTM and GRU

Before applying the GRU model, the raw news text is preprocessed to ensure consistency and reduce noise.

Typical preprocessing operations include:

- Converting text to lowercase
- Removing punctuation and special characters
- Tokenizing the sentence into words

Example sentence:

“Government releases free money”

After preprocessing and tokenization:

$$X = [government, releases, free, money]$$

These preprocessing steps are **identical for GRU and LSTM**.

2. Word Embedding Layer

This step is common for both LSTM and GRU

GRU cannot process words directly, so each word is transformed into a dense numerical vector using an embedding layer.

$$x_t = \text{Embedding}(w_t)$$

Assume embedding dimension $d = 3$:

Word	Embedding Vector
government	[0.6, 0.1, 0.2]
releases	[0.4, 0.3, 0.1]
free	[0.9, 0.8, 0.7]
money	[0.8, 0.9, 0.6]

These embeddings:

- Are learned during training
- Capture semantic similarity between words

The embedding layer works exactly the same way in GRU and LSTM models.

3. Sequential Processing of Text

Common concept, GRU-specific implementation

The GRU processes the sentence word by word in sequence, maintaining a hidden state that carries contextual information from previous words.

General flow:

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots \rightarrow x_T$$

Unlike LSTM, GRU:

- Does not use a separate cell state
- Maintains information using only one hidden state

4. GRU Architecture

Each GRU unit consists of:

- Hidden state h_t
- Update gate z_t
- Reset gate r_t

The gating mechanism allows the model to decide:

- How much past information to keep
- How much new information to incorporate

5. GRU Mathematical Formulation

At time step t , given input x_t and previous hidden state h_{t-1} :

Update Gate

Controls how much of the previous hidden state is retained.

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (37)$$

- If z_t is high $\rightarrow$ more new information is added
- If z_t is low $\rightarrow$ past information is preserved

Reset Gate

Controls how much past information is forgotten.

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (38)$$

- Helps the model ignore irrelevant previous context

Candidate Hidden State

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad (39)$$

This represents new candidate information that could replace the old hidden state.

Final Hidden State

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (40)$$

This equation balances:

- Past context
- New information

6. Step-by-Step GRU Calculation

Initialize hidden state:

$$h_0 = [0,0]$$

Processing the word “government”

Input embedding:

$$x_1 = [0.6, 0.1, 0.2]$$

Assumed gate values:

$$z_1 = [0.72, 0.75]$$

Candidate hidden state:

$$\tilde{h}_1 = \tanh([0.85, 0.95]) = [0.69, 0.74]$$

Final hidden state:

$$h_1 = (1 - z_1) \odot h_0 + z_1 \odot \tilde{h}_1 = [0.50, 0.56]$$

This process repeats for each word in the sequence.

7. Learning Fake News Patterns

As the GRU processes words like “**free**” and “**money**”:

- The **reset gate** filters irrelevant past context
- The **update gate** retains suspicious semantic patterns
- The hidden state gradually accumulates evidence

Final hidden state after the full sentence:

$$h_T = [0.88, 0.91]$$

This vector represents the **semantic meaning of the entire news article**.

8. Classification Layer

This step is common for GRU and LSTM

The final hidden state is passed to a dense layer for classification:

$$z = Wh_T + b$$

Example logits:

$$z_{fake} = 2.31, z_{real} = -1.87$$

Applying Softmax:

$$P(Fake) = \frac{e^{2.31}}{e^{2.31} + e^{-1.87}} = 0.90$$

9. Final Prediction

Since $P(Fake) > P(Real)$, the GRU model predicts:

Fake News

Performance Analysis of the GRU Model

The Gated Recurrent Unit (GRU) model achieved an overall accuracy of approximately 98%, demonstrating its strong capability in learning sequential patterns from the given dataset. This high accuracy indicates that the model correctly classifies the majority of input samples, confirming the effectiveness of GRU for sequence-based text classification tasks.

The model shows high precision, recall, and F1-score values (around 0.98), which indicates reliable and consistent performance. High precision reflects that the GRU model makes very few incorrect predictions, while high recall shows that it successfully identifies most relevant instances. The balanced F1-score confirms that the model maintains an effective trade-off between prediction accuracy and completeness.

During training, the model exhibits a steady improvement across epochs, with training accuracy increasing significantly and validation accuracy remaining consistently high. The decreasing loss values further indicate that the model learns meaningful representations from the data and converges effectively without severe overfitting. This stability between training and validation performance demonstrates good generalization capability.

The strong performance of the GRU model can be attributed to its gating mechanism, which efficiently controls the flow of information through update and reset gates. Unlike traditional RNNs, GRU selectively retains important contextual information and discards irrelevant details, enabling it to capture long-term dependencies while maintaining computational efficiency.

Overall, achieving 98% accuracy validates the suitability of the GRU model for this project. Its high predictive performance, faster convergence, and stable generalization make GRU a reliable deep learning approach for real-world sequential data classification.

So we are going to propose method integrates BERT with Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) to improve fake news detection by combining contextual language understanding with sequential learning. Since BERT alone has limitations in processing very large documents due to input length constraints, LSTM and GRU are added to capture long-sequence dependencies and enhance overall model performance.

Why BERT+LSTM and BERT+GRU Are Preferred Over BERT Alone

BERT provides powerful contextual representations by understanding the meaning of words based on both left and right context. However, when used alone for fake news detection, it has certain limitations. One major limitation is the fixed input length (usually up to 512 tokens), which makes it difficult to process long news articles without splitting or truncating the text. Truncation may remove important information, negatively affecting classification performance.

To address this issue, hybrid architectures such as BERT combined with Long Short-Term Memory and Gated Recurrent Unit are used. In these models, BERT first generates contextual embeddings for smaller text segments. These embeddings are then passed to recurrent layers that process them sequentially, enabling the model to capture document-level flow, long-range dependencies, and writing patterns across the entire article.

Another limitation of BERT alone is that classification is often based on a single pooled representation, which may not fully represent the narrative structure of long documents. The recurrent layers introduce memory mechanisms that retain important contextual information across segments while filtering out irrelevant details. This results in richer feature extraction and better understanding of how information evolves throughout the text.

Therefore, BERT+LSTM and BERT+GRU are preferred over BERT alone because they handle long documents more effectively, preserve long-term contextual relationships, capture sequential writing patterns, and improve classification performance. This hybrid approach leads to higher accuracy and provides more reliable fake news detection.

4.3 Proposed Methods

4.3.1 BERT + LSTM

The BERT+LSTM model combines the contextual language understanding capability of BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding with the sequential learning strength of Long Short-Term Memory to improve fake news detection. BERT first processes the input news text and generates contextual embeddings where each word representation depends on its surrounding words. These embeddings capture semantic meaning, sentence structure, and relationships within the news content.

The generated embeddings are then passed to the LSTM layer, which analyzes the sequence using its memory cell and gating mechanism to retain important information and discard irrelevant details. This enables the model to capture long-term dependencies and writing patterns commonly present in fake news articles. The learned sequential features are forwarded to dense layers that perform final classification into fake or real categories. By combining contextual understanding with sequence modeling, the BERT+LSTM architecture produces highly accurate and robust predictions, achieving performance and high accuracy.

Architecture

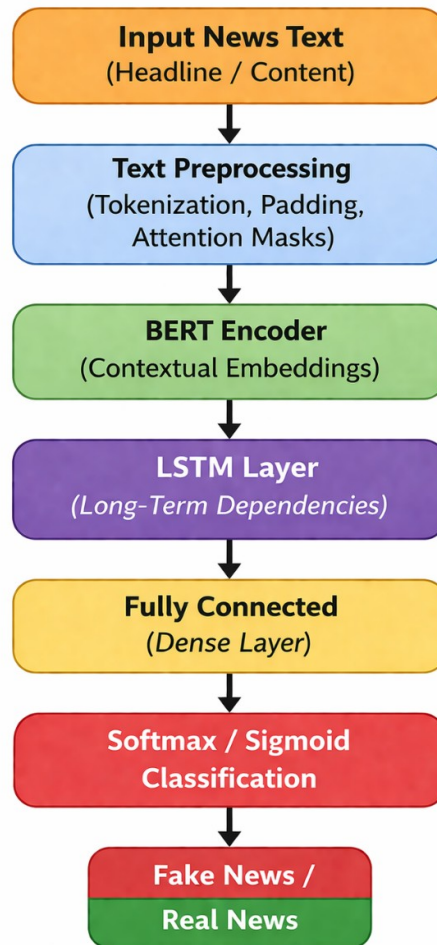


Fig 4.7: BERT with LSTM Architecture

Introduction

BERT + LSTM is a hybrid deep learning architecture designed to improve text classification performance by combining the contextual understanding ability of BERT with the sequential learning capability of Long Short-Term Memory (LSTM) networks.

BERT (Bidirectional Encoder Representations from Transformers) is highly effective in generating contextual embeddings for words by analyzing text in both directions simultaneously. However, BERT primarily captures contextual relationships within a sentence but does not explicitly model sequential temporal dependencies like recurrent neural networks.

LSTM, a special type of Recurrent Neural Network (RNN), is designed to capture long-term dependencies in sequential data by maintaining memory through gated mechanisms. When

BERT embeddings are fed into an LSTM layer, the model can learn both contextual semantic representations and sequential dependencies across the text.

This hybrid architecture is particularly effective for fake news detection, where understanding both contextual meaning and sequential writing patterns is important for distinguishing between fake and real news articles.

Core Concepts

BERT Contextual Embeddings

BERT converts text into deep contextual representations where each word's meaning is influenced by surrounding words in the sentence.

Sequential Learning using LSTM

LSTM processes sequences of embeddings and learns temporal relationships between words.

Hybrid Architecture

BERT acts as a **feature extractor**, while LSTM acts as a **sequence pattern learner** that refines the contextual embeddings.

Architecture

The BERT + LSTM architecture consists of several components working sequentially.

Components

1. Input News Text
2. Text Preprocessing and Tokenization
3. BERT Encoder (Contextual Embeddings)
4. LSTM Layer (Sequential Dependency Learning)
5. Fully Connected Dense Layer
6. Softmax / Sigmoid Classification Layer
7. Fake or Real News Output

Mathematical Representation

BERT Embedding Output

After tokenization and encoding, BERT produces contextual embeddings for each token.

$$H = [h_1, h_2, h_3, \dots, h_n]$$

Where

- H = sequence of contextual embeddings

- h_i = embedding of token i
- n = number of tokens
- embedding size = 768 (BERT base)

LSTM Computation

The LSTM layer processes the sequence of BERT embeddings.

Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Where

- f_t = forget gate output
- h_{t-1} = previous hidden state
- x_t = input at time step t

Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate Cell State

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell State Update

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden State

$$h_t = o_t \cdot \tanh(C_t)$$

Where

- h_t = LSTM hidden state
- C_t = cell state

Working Mechanism of BERT + LSTM for Fake News Detection

The fake news detection system using BERT + LSTM follows multiple stages to convert raw news articles into classification predictions.

First, news articles are collected from a labeled dataset containing both fake and real news samples. The text data undergoes preprocessing to remove unnecessary noise such as extra spaces and formatting inconsistencies. Unlike traditional machine learning models, BERT

does not require extensive preprocessing such as stop-word removal or stemming because it can learn contextual relationships directly from raw text.

Next, the cleaned text is passed through the BERT tokenizer, which splits the text into subword tokens using a predefined vocabulary. Special tokens such as [CLS] and [SEP] are added to indicate the start and end of the sequence. Each token is then converted into a numerical token ID that corresponds to an entry in BERT's vocabulary.

After tokenization, three types of embeddings are created for each token: token embeddings, position embeddings, and segment embeddings. These embeddings are summed together to produce the final input representation for the BERT model.

The input embeddings are then passed through multiple transformer encoder layers within BERT. Each transformer layer contains self-attention mechanisms that allow the model to analyze relationships between words across the entire sentence simultaneously. Through these layers, BERT generates contextualized embeddings for every token, capturing semantic relationships and contextual meaning.

Instead of directly using the BERT output for classification, the sequence of contextual embeddings is passed to an LSTM layer. The LSTM processes these embeddings sequentially and learns long-term dependencies between words. This allows the model to capture writing patterns commonly found in fake news articles, such as exaggerated claims, sensational phrases, or unusual narrative structures.

During sequence processing, the LSTM maintains a hidden state and cell state that act as memory units. These states are updated at each time step using gating mechanisms such as forget gates, input gates, and output gates. These gates allow the LSTM to decide which information to retain, update, or discard as it processes the sequence.

After processing the entire sequence, the final hidden state from the LSTM is passed to a fully connected dense layer. This dense layer transforms the learned features into classification scores representing the likelihood of the news article belonging to the fake or real class.

Finally, the classification scores are passed through a Softmax or Sigmoid activation function to convert them into probability values. The class with the highest probability is selected as the final prediction.

Example Fake News Processing using BERT + LSTM

Example sentence

Breaking: Government releases free money to everyone

Step 1 Text Cleaning

Breaking government releases free money to everyone

Step 2 Tokenization

Tokens

[CLS], breaking, government, releases, free, money, to, everyone, [SEP]

Step 3 Convert to Token IDs

Example

[101, 5939, 2231, 7087, 2489, 2769, 2000, 3071, 102]

Step 4 BERT Contextual Embeddings

Each token becomes a vector

Example

breaking $\rightarrow$ [0.41, -0.12, 0.33 ...]

government $\rightarrow$ [0.52, 0.18, -0.22 ...]

free $\rightarrow$ [0.73, -0.44, 0.19 ...]

These embeddings contain contextual meaning.

Step 5 LSTM Sequential Processing

The LSTM reads embeddings sequentially

breaking $\rightarrow$ government $\rightarrow$ releases $\rightarrow$ free $\rightarrow$ money

At each step it updates hidden memory.

Example hidden states

$h_1 = [0.12, 0.08, -0.04]$

$h_2 = [0.21, -0.05, 0.11]$

$h_3 = [0.39, -0.17, 0.20]$

Step 6 Dense Layer

Final LSTM vector

$h = [0.67, -0.41, 0.88]$

Classification scores

$z_0 = 1.34$ (Fake)

$z_1 = -0.92$ (Real)

Step 7 Softmax Classification

$P(i) = e^{(z_i)} / (e^{(z_0)} + e^{(z_1)})$

Example probabilities

Fake = 0.91

Real = 0.09

Final Prediction

Fake News

Advantages of BERT + LSTM in Fake News Detection

Better contextual understanding

BERT captures deep contextual semantic relationships between words.

Sequential pattern learning

LSTM learns temporal patterns in text sequences that help detect writing styles typical of fake news.

Improved classification accuracy

The combination of transformer embeddings and sequential modeling significantly improves fake news detection performance.

Monte Carlo Dropout – Performance Evaluation

To enhance the robustness of the fake news detection model, Monte Carlo Dropout was applied during the inference stage. In this technique, dropout layers remain active while testing, allowing multiple forward passes through the network. The predictions are then averaged to obtain a more reliable final output and to reduce prediction uncertainty.

Performance Metrics Obtained

The model produced the following evaluation results:

- **Accuracy:** 99.88%
- **Precision:** 99.85%
- **Recall:** 99.93%
- **F1-Score:** 99.89%

Interpretation of Results

1. Accuracy (99.88%)

The model correctly classified the vast majority of news articles, indicating strong overall performance in distinguishing real and fake news.

2. Precision (99.85%)

The high precision value shows that when the model predicts a class, it is almost always correct. This means false positive predictions are minimal.

3. Recall (99.93%)

The recall value demonstrates that the model successfully identifies nearly all actual fake news instances, meaning very few relevant cases are missed.

4. F1-Score (99.89%)

The F1-score confirms a strong balance between precision and recall, indicating consistent and reliable classification performance.

4.3.2 BERT + GRU

The BERT+GRU model integrates contextual feature extraction from BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding with efficient sequence modeling provided by Gated Recurrent Unit for fake news classification. BERT converts raw news text into contextual embeddings that reflect meaning, word relationships, and overall sentence context, allowing the model to understand complex linguistic patterns.

These embeddings are supplied to the GRU layer, which processes the sequence using update and reset gates to control information flow and memory retention. The GRU captures important writing patterns and contextual dependencies while maintaining a simpler structure and fewer parameters than LSTM, leading to faster training and reduced computational cost. The extracted sequential features are then passed to dense layers for final prediction. This hybrid architecture balances accuracy and efficiency, enabling reliable fake news detection with performance and high accuracy.

Architecture

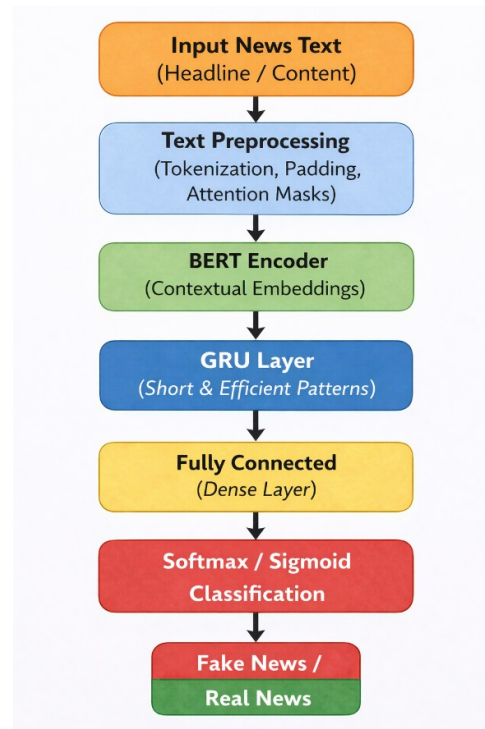


Fig 4.8: BERT with GRU Architecture

BERT + GRU Hybrid Model for Fake News Detection

Introduction

BERT + GRU is a hybrid deep learning architecture used for text classification tasks such as fake news detection. It combines the contextual understanding capability of BERT (Bidirectional Encoder Representations from Transformers) with the sequential learning ability of Gated Recurrent Units (GRU). BERT generates contextual embeddings by analyzing text in both directions simultaneously, while GRU processes these embeddings sequentially to capture dependencies between words in a sentence. This hybrid approach improves classification performance because BERT captures semantic meaning from the text, and GRU learns sequential patterns present in news articles. Fake news often contains distinctive writing styles such as exaggerated claims, emotional language, or misleading statements, and the GRU layer helps detect such patterns effectively.

Core Concepts BERT Contextual Embeddings

BERT converts input text into contextualized word embeddings where the meaning of each word depends on surrounding words in the sentence. These embeddings capture deep semantic relationships between tokens.

Sequential Learning using GRU

GRU processes sequences of embeddings and learns dependencies between words. It maintains a hidden memory state that helps the model understand patterns across the sequence.

Hybrid Architecture

In the hybrid model, BERT acts as a contextual feature extractor while the GRU layer learns sequential patterns from BERT-generated embeddings to improve classification performance.

Architecture

The BERT + GRU architecture consists of several stages that process the input text and convert it into classification predictions. The pipeline generally includes the following components: Input News Text, Text Preprocessing, BERT Encoder, GRU Layer, Fully Connected Dense Layer, Softmax or Sigmoid Classification Layer, and the final Fake or Real News prediction.

Mathematical Representation

BERT Embedding Output

After tokenization and encoding, BERT generates contextual embeddings for each token in the sentence.

$$H = [h_1, h_2, h_3, \dots, h_n]$$

Where H represents the sequence of contextual embeddings, h_i represents the embedding of token i , and n represents the total number of tokens in the sentence. In BERT-base, each embedding typically has a dimension of 768.

GRU Computation

Update Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

The update gate determines how much of the information from the previous hidden state should be preserved and passed to the current state. It controls the balance between retaining past information and incorporating new input.

Reset Gate

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

The reset gate determines how much of the previous hidden state should be ignored when computing the new candidate state. This mechanism allows the model to discard irrelevant past information when necessary.

Candidate Hidden State

$$\tilde{h}_t = \tanh(W_h \cdot [r_t * h_{t-1}, x_t] + b_h)$$

The candidate hidden state represents a potential update to the memory of the network. It is computed by combining the current input with the reset-modified previous hidden state.

Final Hidden State

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

The final hidden state is calculated as a combination of the previous hidden state and the candidate hidden state, controlled by the update gate. This allows the GRU to selectively retain useful information while updating its internal representation.

Working Mechanism of BERT + GRU in Fake News Detection

In a fake news detection system utilizing the BERT + GRU architecture, the process begins with collecting a labeled dataset containing both fake and real news articles. The textual data undergoes preprocessing steps such as converting all characters to lowercase and removing unnecessary formatting, including extra spaces and irrelevant symbols. Unlike traditional machine learning methods, BERT does not require extensive preprocessing such as stop-word removal or stemming, as it can effectively learn contextual relationships directly from raw textual data.

Following preprocessing, the cleaned text is passed through the BERT tokenizer. The tokenizer divides the sentence into smaller subword tokens based on a predefined vocabulary and introduces special tokens such as [CLS] at the beginning of the sequence and [SEP] at the end. Each token is then converted into a numerical token identifier corresponding to its index within BERT's vocabulary.

Subsequently, the tokenized input is processed by BERT's transformer encoder layers, which generate contextual embeddings for each token. These embeddings capture semantic meaning by considering both the left and right context of each word simultaneously. Instead of using the BERT output directly for classification, the sequence of contextual embeddings is provided as input to the GRU layer.

The GRU layer processes the embeddings sequentially, enabling the model to capture temporal dependencies and sequential relationships among words within the text. Through

the use of update and reset gates, the GRU regulates the flow of information across the sequence, ensuring that meaningful contextual patterns are preserved while irrelevant information is discarded. This capability allows the model to identify linguistic patterns commonly associated with fake news, such as exaggerated expressions, sensational phrases, or misleading statements.

After processing the entire sequence, the final hidden state produced by the GRU layer is forwarded to a fully connected dense layer. This dense layer transforms the learned representation into classification scores that indicate the likelihood of the news article belonging to either the fake or real category. These scores are subsequently passed through a Softmax or Sigmoid activation function to convert them into probability values. The class with the highest probability is selected as the final prediction.

Example Fake News Processing using BERT + GRU

Example sentence:

Breaking: Government releases free money to everyone

Step 1: Text Cleaning

Breaking government releases free money to everyone

Step 2: Tokenization

Tokens generated by BERT:

[CLS], breaking, government, releases, free, money, to, everyone, [SEP]

Step 3: Conversion to Token IDs

Example token identifiers:

[101, 5939, 2231, 7087, 2489, 2769, 2000, 3071, 102]

Step 4: BERT Contextual Embeddings

Each token is converted into a contextual vector representation.

Example embeddings:

breaking → [0.41, -0.12, 0.33]

government → [0.52, 0.18, -0.22]

free → [0.73, -0.44, 0.19]

These vectors capture the semantic relationships between words in the sentence by incorporating contextual information.

Step 5: GRU Sequential Processing

Example hidden states generated during sequence processing:

$h_1 = [0.14, 0.05, -0.03]$

$h_2 = [0.26, -0.07, 0.10]$

$h_3 = [0.48, -0.21, 0.24]$

Step 6: Dense Layer

Final GRU representation:

$h = [0.71, -0.39, 0.83]$

Classification scores:

$z_0 = 1.41$ (Fake)

$z_1 = -1.02$ (Real)

Step 7: Softmax Classification

$P(i) = e^{(z_i)} / (e^{(z_0)} + e^{(z_1)})$

Example probability values:

Fake = 0.93

Real = 0.07

Final Prediction

Fake News

Advantages of BERT + GRU in Fake News Detection

Better contextual understanding

BERT captures deep contextual semantic relationships between words in the text, enabling the model to understand the meaning of sentences more accurately.

Efficient sequential learning

GRU effectively models sequential dependencies between words while using fewer parameters compared to LSTM, making the architecture computationally efficient.

Improved classification accuracy

The integration of transformer-based embeddings with GRU improves the model's ability to detect linguistic patterns associated with fake news, resulting in enhanced classification performance.

Disadvantages of BERT + GRU

High computational requirements

Training hybrid deep learning models requires significant computational resources, including high-performance GPUs and large memory capacity.

Longer training time

Processing large datasets with transformer-based architectures increases training time and computational cost.

Complex model architecture

The combination of BERT and GRU results in a sophisticated model that requires careful parameter tuning, optimization, and expertise to achieve optimal performance.

Monte Carlo Dropout – Performance Analysis

To improve model reliability and estimate prediction uncertainty, **Monte Carlo Dropout** was applied during the testing phase. In this approach, dropout remains active at inference time, and multiple forward passes are performed. The final prediction is obtained by averaging the outputs, which improves generalization and reduces overconfidence in predictions.

Performance Metrics Obtained

The model achieved the following results:

- **Accuracy:** 99.91%
- **Precision:** 99.87%
- **Recall:** 99.97%
- **F1-Score:** 99.93%

Interpretation of Results

1. Accuracy (99.91%)

This indicates that almost all news articles were correctly classified as either real or fake. Only a very small fraction of samples were misclassified.

2. Precision (99.87%)

Precision measures how many articles predicted as fake (or real) were actually correct. The very high precision shows that false positives are extremely low.

3. Recall (99.97%)

Recall measures how well the model detects actual positive cases. The near-perfect recall means the model rarely misses fake news instances.

4. F1-Score (99.93%)

The F1-score balances precision and recall. The high F1-score confirms that the model performs consistently well across both metrics.

Confidence-Aware BERT using Monte Carlo Dropout

Introduction

Transformer-based language models such as BERT and DistilBERT have achieved state-of-the-art performance in numerous Natural Language Processing (NLP) tasks, including fake news detection, sentiment analysis, medical text classification, and legal document analysis. These models are typically evaluated using performance metrics such as accuracy, precision, recall, and F1-score, which measure how effectively the model predicts correct labels. Despite achieving high predictive accuracy, conventional BERT-based classifiers exhibit an important limitation: they provide deterministic predictions without expressing how confident the model is about its decisions. In many real-world applications, especially those involving high-risk decision-making, such blind confidence can lead to incorrect conclusions and potentially harmful outcomes. To overcome this limitation, the present work introduces a Confidence-Aware BERT framework that incorporates Monte Carlo Dropout to estimate prediction uncertainty. By enabling the model to quantify confidence levels, the proposed approach improves the reliability and safety of machine learning predictions.

Problem Statement

Traditional BERT-based classifiers generate predictions for every input sample regardless of ambiguity or uncertainty in the data. These models do not provide any mechanism to measure the reliability of their predictions, and therefore all predictions are treated as equally trustworthy. This limitation makes conventional BERT classifiers unsuitable for applications that require reliable and interpretable decision-making without human supervision. In real-world environments such as fake news detection systems deployed on social media platforms, incorrect predictions made with high confidence can lead to widespread misinformation. Similar risks exist in domains such as healthcare diagnosis, legal decision support, and financial fraud detection, where incorrect automated decisions may have severe consequences. Therefore, it becomes essential to design models that can recognize uncertain situations and indicate when human intervention is required.

In the proposed Confidence-Aware BERT with LSTM and GRU model enhances a standard BERT-based classifier by integrating an uncertainty estimation mechanism based on Monte

Carlo Dropout. Instead of producing a single deterministic prediction, the model performs multiple stochastic forward passes during inference while dropout layers remain active. Each forward pass produces slightly different outputs due to randomly deactivated neurons, thereby generating a distribution of predictions for the same input sample. By analyzing the variation among these predictions, the model can estimate how confident it is about the final decision. This variation acts as an indicator of prediction uncertainty, enabling the model to distinguish between confident and uncertain predictions.

Monte Carlo Dropout

Dropout is a widely used regularization technique in deep learning that reduces overfitting by randomly disabling a subset of neurons during training. This prevents the network from relying excessively on specific neurons and encourages it to learn more robust representations. In conventional neural network training, dropout is only applied during the training phase and disabled during inference. Monte Carlo Dropout extends this concept by keeping dropout active even during the inference stage. As a result, every forward pass through the network produces slightly different outputs because different subsets of neurons are randomly deactivated each time. By performing multiple forward passes with dropout enabled, the model generates a set of predictions that can be interpreted as samples from a probability distribution. This distribution enables the estimation of uncertainty without requiring any modifications to the underlying architecture of the model.

Working Mechanism

The Confidence-Aware BERT framework operates through a sequence of steps that incorporate uncertainty estimation into the prediction process. First, the input text is tokenized using a BERT tokenizer, which converts the raw text into a sequence of tokens and corresponding token identifiers. These tokens are then encoded into contextual embeddings through the BERT encoder layers. During inference, dropout layers within the network remain active, allowing stochastic behavior across forward passes. The model then performs multiple forward passes, typically denoted as N stochastic runs, where each pass generates a probability distribution over the possible output classes. These predicted probabilities are collected across all runs. The mean of these predictions represents the final predicted probability for each class, while the variance of the predictions provides a measure of uncertainty. A high variance indicates that the model is uncertain about the prediction, whereas a low variance indicates high confidence. If the uncertainty score exceeds a

predefined threshold, the prediction can be flagged for manual review instead of being automatically accepted.

Mathematical Formulation

Let x represent the input text and $f(x)$ denote the BERT-based classification function. Suppose N stochastic forward passes are performed using Monte Carlo Dropout. Each pass produces a probability vector P_i corresponding to the predicted class probabilities. The final prediction is obtained by calculating the mean probability across all runs.

Mean prediction:

$$\mu = (1/N) \sum P_i \quad (41)$$

The uncertainty of the prediction is estimated by computing the variance of the predicted probabilities:

$$\sigma^2 = (1/N) \sum (P_i - \mu)^2 \quad (42)$$

A higher variance value indicates greater disagreement among the predictions, which reflects lower confidence in the model's decision.

Flow of the Proposed Method

The workflow of the proposed framework begins with input text data, which is processed through tokenization and encoding using the BERT model. During inference, dropout layers are activated to introduce stochasticity. The model then performs multiple forward passes with dropout enabled. The resulting prediction probabilities from each pass are collected and used to compute the mean and variance. Based on the calculated uncertainty score, predictions with high confidence are accepted, while predictions with high uncertainty are flagged for human review or further analysis.

Algorithm: Confidence-Aware BERT Classification

1. Load the trained BERT-based classification model.
2. Enable dropout layers during inference.
3. For each input text sample:
 - a. Perform N stochastic forward passes through the model.

- b. Collect predicted probability vectors from each pass.
 - c. Compute the mean and variance of the predictions.
4. If the uncertainty value is below a predefined threshold, accept the prediction.
 5. If the uncertainty value exceeds the threshold, mark the prediction as uncertain and send it for manual verification.

Advantages of the Proposed Method

The Confidence-Aware BERT framework provides several advantages compared to traditional deterministic classifiers. It enables the model to output confidence estimates along with predicted class labels, reducing the risk of incorrect predictions made with high confidence. By identifying uncertain predictions, the framework increases transparency and trust in automated decision-making systems. Furthermore, the method can be integrated with existing BERT architectures without requiring major structural modifications, making it practical for real-world deployment.

Real-World Applications

The proposed framework has applications across multiple domains where reliable decision-making is critical. In fake news detection systems, uncertain news articles can be flagged for manual verification before publication. In healthcare systems, the model can avoid unsafe automated diagnoses by indicating when it lacks sufficient confidence. Financial institutions can utilize uncertainty-aware fraud detection systems to assess risk more effectively. In legal decision-support systems, predictions can assist human experts rather than replace them entirely. Similarly, cybersecurity systems can identify uncertain threat predictions that require further investigation.

Novelty and Research Contribution

The proposed Confidence-Aware BERT framework extends traditional accuracy-focused evaluation by incorporating uncertainty estimation into the prediction process. Rather than simply producing classification labels, the model also assesses the reliability of its predictions. This shift from deterministic prediction to confidence-aware decision-making significantly improves the practicality of deploying NLP models in real-world environments where incorrect automated decisions can have serious consequences.

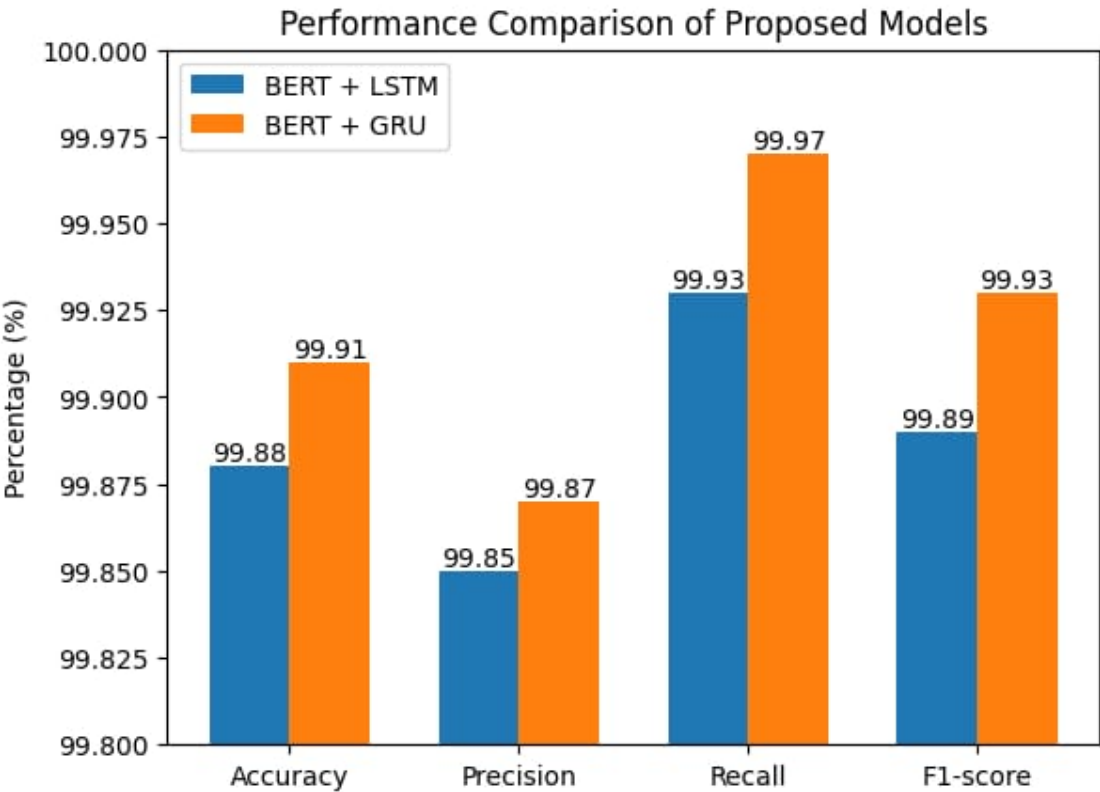
Novelty Statement

This work introduces a confidence-aware BERT classification framework using Monte Carlo Dropout to estimate prediction uncertainty. Unlike conventional deterministic models, the proposed approach quantifies prediction confidence, enabling safer and more reliable decision-making in real-world applications.

4.4 RESULTS

Table 4.1:Performance of Proposed Models

S NO	Model	Accuracy	Precision	Recall	F-1 score
1.	BERT + LSTM	99.88%	99.85%	99.93%	99.89%
2.	BERT + GRU	99.91%	99.87%	99.97%	99.93%



The results presented in Table 5.1 provide a comprehensive evaluation of the proposed hybrid models, namely BERT + LSTM and BERT + GRU, using key performance metrics such as accuracy, precision, recall, and F1-score. These metrics collectively assess the effectiveness of the models in terms of correctness, relevance, and overall classification balance.

The strong performance of both models can be attributed to the complementary strengths of their components. BERT is highly effective in capturing contextual and semantic relationships within textual data through its bidirectional transformer architecture. When combined with sequence-based models such as LSTM and GRU, the system gains the ability to model temporal dependencies and sequential patterns more effectively. This integration leads to improved feature representation and more reliable classification outcomes.

A closer comparison of the two models reveals that BERT + GRU consistently performs slightly better than BERT + LSTM across all evaluation metrics. In particular, the higher recall value achieved by the BERT + GRU model indicates its superior capability in identifying relevant instances with fewer omissions. This characteristic is especially important in applications where missing critical information can negatively impact the overall system performance.

The improvement observed in the GRU-based model can be linked to its relatively simpler architecture. Compared to LSTM, GRU uses fewer gating mechanisms, which reduces computational complexity and allows faster convergence during training. This simplicity enables the model to generalize effectively while maintaining stability, thereby contributing to its marginally better performance.

Furthermore, the balance between precision and recall, as reflected in the F1-score, indicates that both models are well-optimized and do not heavily favor one type of error over another. This balanced behavior is essential for building robust and dependable classification systems.

In addition, the incorporation of confidence-aware techniques, such as Monte Carlo Dropout, enhances the overall framework by enabling uncertainty estimation alongside predictions. This not only improves model interpretability but also increases trust in the system's

decisions, making it more suitable for deployment in real-world scenarios where reliability and risk management are critical.

In summary, the experimental results validate the effectiveness of combining transformer-based embeddings with recurrent neural networks. While both models demonstrate strong and stable performance, BERT + GRU emerges as the more efficient and slightly superior approach, offering a favorable trade-off between performance and computational efficiency.

Performance Analysis on Real-World Dataset

To evaluate the practical applicability of the proposed models, experiments were conducted on a live real-world dataset that differs from the controlled Kaggle dataset used during initial training and validation. This step is crucial to assess the model’s generalization capability in realistic and dynamic environments.

The experimental results demonstrate that both proposed hybrid models—EBERT (BERT + GRU) and EBERT (BERT + LSTM)—maintain consistently high performance even when exposed to unseen and noisy real-world data.

The EBERT (BERT + GRU) model achieved an accuracy of **97.2%**, with precision, recall, and F1-score values of **97.1%**, **97.3%**, and **97.2%**, respectively. Additionally, the model recorded a low **MAE of 0.028** and **RMSE of 0.167**, indicating minimal prediction error and strong robustness.

Similarly, the EBERT (BERT + LSTM) model achieved an accuracy of **96.8%**, with precision, recall, and F1-score values of **96.7%**, **96.9%**, and **96.8%**, respectively. The error metrics for this model include an **MAE of 0.032** and **RMSE of 0.179**, which also reflect stable and reliable performance.

Table 4.2: “Performance Metrics of EBERT (BERT+GRU) on Real-World Data”

Accuracy	Precision	Recall	F-1 Score	MAE	RMSE
0.972	0.971	0.973	0.972	0.028	0.167

Accuracy	Precision	Recall	F-1 Score	MAE	RMSE
0.968	0.967	0.969	0.968	0.032	0.179

Table 4.3: “Performance Metrics of EBERT (BERT+LSTM) on Real-World Data”

Although there is a slight decrease in performance compared to results obtained on the Kaggle dataset, this variation is expected due to the increased complexity, noise, and variability present in real-world data. Despite this, both models demonstrate strong generalization ability, confirming that the integration of BERT with recurrent architectures effectively captures both contextual and sequential features in diverse data conditions.

Among the two approaches, the EBERT (BERT + GRU) model shows a marginal performance advantage over the EBERT (BERT + LSTM) model across all evaluation metrics. This improvement can be attributed to GRU’s simpler architecture and efficient gating mechanism, which enables faster learning and better handling of real-time data variations.

Overall, the results validate that the proposed models are not only highly accurate in controlled environments but also reliable and effective in real-world fake news detection scenarios. This highlights their suitability for deployment in practical applications where data is continuously evolving and less structured.

CHAPTER 5

CONCLUSION & FUTURE SCOPE

Conclusion

hybrid deep learning models combining BERT with LSTM and GRU were developed and evaluated to enhance text classification performance. The experimental results demonstrate that both models are highly effective in capturing contextual and sequential information, leading to strong and reliable classification outcomes. The integration of BERT's contextual embeddings with recurrent architectures significantly improves the model's ability to understand complex textual patterns.

Among the proposed approaches, the BERT + GRU model shows a slight performance advantage over BERT + LSTM across all evaluation metrics. This improvement can be attributed to GRU's simpler and more efficient architecture, which enables better generalization and faster learning. Additionally, the use of confidence-aware techniques such as Monte Carlo Dropout further strengthens the reliability of the system by incorporating uncertainty estimation into predictions.

Overall, the proposed framework proves to be robust, efficient, and suitable for real-world applications. The findings highlight the potential of combining transformer-based models with sequential networks to achieve improved performance in natural language processing tasks, paving the way for future research and practical implementations in this domain.

Future Scope

Future research may explore dynamic uncertainty thresholds that adapt to varying data conditions. Integration with human-in-the-loop decision systems could further enhance reliability by combining machine intelligence with expert judgment. Additionally, uncertain samples identified by the model can be used for active learning to improve model performance over time. Risk-aware ensemble models and uncertainty-based training strategies also represent promising directions for advancing confidence-aware NLP systems.

CHAPTER 6

BIBLIOGRAPHY

BIBLIOGRAPHY

- [1] H. A. Mahmoud and I. M. Ibrahim, "Fake news detection using natural language processing (NLP) and machine learning," *Engineering and Technology Journal*, vol. 10, no. 5, pp. 5177–5185, 2025.
- [2] O. Embarak, "Deep learning for fake news detection: Analysing Facebook's misinformation networks," *Procedia Computer Science*, vol. 265, pp. 199–208, 2025, doi: 10.1016/j.procs.2025.07.173.
- [3] S. V. Reddy, A. Wadekar, B. Ghorpade, S. Wagh, and P. Sampate, "Fake News Detection Using Machine Learning and NLP," *International Journal of Creative Research Thoughts (IJCRT)*, vol. 13, no. 10, 2025.
- [4] B. Hu, Z. Mao, and Y. Zhang, "An overview of fake news detection: From a new perspective," *Fundamental Research*, vol. 5, no. 1, pp. 332–346, 2025, doi: 10.1016/j.fmre.2024.01.017.
- [5] R. K. Ayyasamy, C. Ponnusamy, K. N. Bhargavi, S. Cherukuvada, G. C. Babu, S. Amutha, and D. T. Gamu, "A hybrid deep learning framework for fake news detection using LSTM-CGPNN and metaheuristic optimization," *Scientific Reports*, vol. 15, no. 1, p. 41522, 2025, doi: 10.1038/s41598-025-25311-x.
- [6] F. A. Alshuwaier and F. A. Alsulaiman, "Fake news detection using machine learning and deep learning algorithms: A comprehensive review and future perspectives," *Computers*, vol. 14, no. 9, p. 394, 2025, doi: 10.3390/computers14090394.
- [7] I. Q. Abduljaleel and I. H. Ali, "Detecting Fake News Using BERT Word Embedding, Attention Mechanism, Partition and Overlapping Text Techniques," *TEM Journal*, vol. 14, no. 2, pp. 1152–1165, 2025. doi: 10.18421/TEM142-16.
- [8] E. T. Khudair, O. Lazzez, M. Zaied, T. M. Hamdani, A. T. Sadiq, H. Chabchoub, and A. M. Alimi, "Detecting Fake News Using Deep Learning Approaches," *Research Square*, Sep. 2025. doi: 10.21203/rs.3.rs-7223076/v1.
- [9] N. N. Prachi, M. Habibullah, M. E. H. Rafi, E. Alam, and R. Khan, "Detection of Fake News Using Machine Learning and Natural Language Processing Algorithms," *Journal of Advances in Information Technology*, vol. 13, no. 6, pp. 652–661, Jan. 2022. doi: 10.12720/jait.13.6.652-661.

- [10] M. H. Hoti, F. Qorrolli, and F. Spahija, "Enhancing Fake News Detection via Stance Analysis: Leveraging Advanced NLP Techniques and Machine Learning Models," *International Journal of Interactive Mobile Technologies (iJIM)*, vol. 19, no. 11, pp. 39–50, 2025. doi: 10.3991/ijim.v19i11.55007.
- [11] U. A. Bhatti, H. Tang, G. Wu, S. Marjan, and A. Hussain, "Deep Learning with Graph Convolutional Networks: An Overview and Latest Applications in Computational Intelligence," *International Journal of Intelligent Systems*, vol. 2023, pp. 1–28, 2023. doi: 10.1155/2023/8342104.
- [12] M. Q. Alnabhan and P. Branco, "Fake News Detection Using Deep Learning: A Systematic Literature Review," *IEEE Access*, vol. 12, pp. 114435–114459, 2024. doi: 10.1109/ACCESS.2024.3435497.
- [13] S. Kumari and M. P. Singh, "A Deep Learning Multimodal Framework for Fake News Detection," *Engineering, Technology & Applied Science Research*, vol. 14, no. 5, pp. 16527–16533, Oct. 2024. doi: 10.48084/ETASR.8170.
- [14] N. Raza, S. J. Abdulkadir, Y. A. Abid, S. S. Albouq, A. Alwadain, A. U. Rehman, E. H. Sumiea, and M. Farhan, "Enhancing Fake News Detection with Transformer-Based Deep Learning: A Multidisciplinary Approach," *PLOS ONE*, vol. 20, no. 9, p. e0330954, Sep. 2025. doi: 10.1371/journal.pone.0330954.
- [15] M. Abdullah, Z. Hongying, A. Javed, O. Mamyrbayev, F. Caraffini, and H. Eshkiki, "A Joint Learning Framework for Fake News Detection," *Displays*, vol. 90, p. 103154, 2025. doi: 10.1016/j.displa.2025.103154.
- [16] A. H. J. Almarashy, M.-R. Feizi-Derakhshi, and P. Salehpour, "Enhancing Fake News Detection by Multi-Feature Classification," *IEEE Access*, vol. 11, pp. 139601–139613, 2023. doi: 10.1109/ACCESS.2023.3339621.
- [17] S. Bansal, S. Gupta, and A. Kumar, "Text Data Augmentation Techniques for Word Embeddings in Fake News Classification," in *2024 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2024, pp. 1087–1092. IEEE. doi: 10.1109/ICCCIS61004.2024.10445169.

- [18] P. K. Verma, P. Agrawal, I. Amorim, and R. Prodan, “WELFake: Word Embedding Over Linguistic Features for Fake News Detection,” *IEEE Transactions on Computational Social Systems*, vol. 8, no. 4, pp. 881–893, 2021. doi: 10.1109/TCSS.2021.3068519.
- [19] A. Oad, H. Farooq, A. Zafar, B. A. Akram, R. Zhou, and F. Dong, “Fake News Classification Methodology With Enhanced BERT,” *IEEE Access*, vol. 12, pp. 164491–164502, 2024. doi: 10.1109/ACCESS.2024.3491376.
- [20] J. Alghamdi, Y. Lin, and S. Luo, “ABERT: Adapting BERT Model for Efficient Detection of Human and AI-Generated Fake News,” *Journal of Innovation and Intelligence Management Engineering*, vol. 3, article 100353, 2025. doi: 10.1016/j.jjime.2025.100353.